

TUNISIAN REPUBLIC
MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH
UNIVERSITY OF SOUSSE
HIGHER INSTITUTE OF COMPUTER SCIENCE
AND COMMUNICATION TECHNOLOGIES



FINAL YEAR PROJECT REPORT

Presented in partial fulfillment of the requirements for the degree of
Bachelor in Computer Engineering

Web/ Smart Mobile Platform for Managing bus Brakes calipers

Prepared by :

Mohamed EL OUARDI

Academic Supervisor :

Mrs. Sonia REZK

Professional Supervisor :

Mr. Makram ERGUEZ

Host Company



Academic year : 2025/2026

TUNISIAN REPUBLIC
MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH
UNIVERSITY OF SOUSSE
HIGHER INSTITUTE OF COMPUTER SCIENCE
AND COMMUNICATION TECHNOLOGIES



FINAL YEAR PROJECT REPORT

Presented in partial fulfillment of the requirements for the degree of
Bachelor in Computer Engineering

Web/ Smart Mobile Platform for Managing bus Brakes calipers

Prepared by :
Mohamed EL OUARDI

Academic year : 2025/2026

Higher Institute of Computer Science and Communication Technologies

ISITCOM

Tél/Fax : +216 73 37 15 71 / +216 73 36 44 11

Signature of Academic Supervisor

Signature of Professional Supervisor

Dedication

I would like to express my sincere gratitude to my dear parents, my family, and all my friends.

Their unwavering support and encouragement throughout my end-of-studies project were invaluable. Their moral support helped me overcome challenges and stay motivated.

I hope that this work meets their expectations.

Mohamed El Ouardi

Acknowledgments

I express my sincere thanks to all the people who contributed to the successful completion of this project.

I would particularly like to thank my academic supervisor, Mrs. Sonia Rezk, for her valuable support and expertise, which enabled me to achieve the objectives set for this project. Her availability during difficult moments, the quality of her supervision, and the relevance of her feedback greatly contributed to the success of this work.

I am also deeply grateful for her understanding and encouragement, which were essential in successfully completing this project.

Mohamed El Ouardi

Table of Contents

List of abbreviations	10
1 General Framework of the Project	14
Introduction	14
1.1 Project Presentation	14
1.2 Presentation of the Host Organization	14
1.2.1 Presentation of the Company	14
1.2.2 Activities and Services	15
1.3 Study of the Existing Situation	15
1.3.1 Project Context	15
1.3.2 Problem Statement	15
1.3.3 Comparative Study	16
1.3.4 Proposed Solution	16
1.4 Project Management Methodology	17
1.4.1 Choice of Methodology	17
1.4.2 SCRUM Methodology	17
1.4.3 SCRUM Principles	17
1.4.4 SCRUM Roles	17
1.4.5 Advantages of SCRUM	18
1.5 Technical Selections	18
1.5.1 Backend Technologies	18
1.5.2 Frontend Technologies	19
1.5.3 Programming Language	19
1.5.4 Database	20
1.5.5 API Implementation	20
1.5.6 Gantt Diagram	20
1.6 Conclusion	21
2 Project Initiation	23
2.1 Introduction	23
2.2 Requirements Capture	23

2.2.1	Identification of Actors	23
2.3	Functional Requirements	24
2.3.1	For Administrator	24
2.3.2	For Agent	24
2.4	Specification of Requirements	24
2.5	Semi-Formal Specification	26
2.5.1	Use Case Diagrams	26
2.5.2	Class Diagram	27
2.5.3	Sprint Planning	28
2.6	MVC Architecture	29
2.7	Conclusion	29
3	Sprint 1: Core System Setup	31
3.1	Introduction	31
3.2	Sprint Backlog: Sprint 1	31
3.3	UML Diagrams Design	32
3.3.1	Use Case Diagram: User Authentication	32
3.3.2	Use Case Diagram: Account Management	33
3.3.3	Use Case Diagram: Calipers Management	34
3.3.4	Use Case Diagram: Data Import Engine	35
3.4	Sequence Diagrams	37
3.4.1	Account Management	37
3.4.2	Calipers Management	38
3.4.3	Data Import Engine	39
3.5	Interface Design and Prototyping	40
3.5.1	User Authentication Interfaces	41
3.5.2	Account Management Interfaces	41
3.5.3	Calipers Management Interfaces	43
3.5.4	Data Import Engine Interfaces	45
3.6	Conclusion	47
4	Sprint 2: Caliper Lifecycle	49
4.1	Introduction	49
4.2	Sprint Backlog: Sprint 2	49
4.3	UML Diagrams Design	50
4.3.1	Use Case Diagram: Caliper Finalization	50
4.3.2	Use Case Diagram: Finished Calipers Management	51
4.3.3	Use Case Diagram: Calipers Overview	52
4.3.4	Use Case Diagram: Calipers Delivery Management	53
4.3.5	Use Case Diagram: Return Handling	54

4.4	Sequence Diagrams	55
4.4.1	Caliper Finalization	55
4.4.2	Finished Calipers Management	57
4.4.3	Calipers Overview	58
4.4.4	Calipers Delivery Management	58
4.4.5	Return Handling	59
4.5	Interface Design and Prototyping	61
4.5.1	Caliper Finalization Interfaces	61
4.5.2	Finished Calipers Management Interfaces	61
4.5.3	Calipers Overview Interfaces	63
4.5.4	Delivery Management Interfaces	63
4.5.5	Return Handling Interfaces	64
4.6	Conclusion	66
5	Sprint 3: Parts Management and AI Integration	68
5.1	Introduction	68
5.2	Sprint Backlog: Sprint 3	68
5.3	UML Diagrams Design	69
5.3.1	Use Case Diagram: Spare Parts Management	69
5.3.2	Use Case Diagram: AI Piece Recognition	70
5.3.3	Use Case Diagram: Voice Command Entry	71
5.4	Sequence Diagrams	72
5.4.1	Spare Parts Management	72
5.4.2	AI Piece Recognition	73
5.4.3	Voice Command Entry	74
5.5	Interface Design and Prototyping	75
5.5.1	Spare Parts Management Interfaces	76
5.5.2	AI Piece Recognition Interfaces	78
5.5.3	Voice Command Entry Interfaces	79
5.6	Conclusion	81
	General Conclusion	82
	Bibliography	84
	Abstract	85

List of Figures

1.1	Company Logo	15
1.2	Application Logo	16
1.3	SCRUM Lifecycle	17
1.4	RESTful API Architecture and Communication Flow	20
1.5	Gantt Diagram	21
2.1	Use Case Diagram	27
2.2	Class Diagram	28
2.3	Sprint Planning	28
2.4	Architecture MVC	29
3.1	Use case "User Authentication"	32
3.2	Use case "Account Management"	33
3.3	Use case "Calipers Management"	34
3.4	Use case "Data Import Engine"	35
3.5	Sequence diagram: "User Authentication"	37
3.6	Sequence diagram: "Account Management"	38
3.7	Sequence diagram: "Calipers Management"	39
3.8	Sequence diagram: "Data Import Engine"	40
3.9	Login Interface - Initial State	41
3.10	Login Interface - Error Handling	41
3.11	Account Creation Interface - Add Form	42
3.12	User Directory View - Selection for Deletion	42
3.13	Deletion Process - Confirmation Modal	43
3.14	Caliper Management - Add Form	43
3.15	Caliper Inventory Overview - Selection Interface	44
3.16	Caliper Status Update Step 1	44
3.17	Caliper Status Update Step 2	44
3.18	Caliper Management - Deletion Confirmation Modal	45
3.19	Data Import - Initial Phase	45
3.20	Data Import - Preview Interface	46

3.21	Data Import - Error Handling	46
3.22	Auto-Sync - Confirmation Modal	47
4.1	Use case "Caliper Finalization"	51
4.2	Use case "Finished Calipers Management"	52
4.3	Use case "Calipers Overview"	53
4.4	Use case "Calipers Delivery Management"	54
4.5	Use case "Return Handling"	55
4.6	Sequence diagram: "Caliper Finalization"	56
4.7	Sequence diagram: "Finished Calipers Management"	57
4.8	Sequence diagram: "Calipers Overview"	58
4.9	Sequence diagram: "Calipers Delivery Management"	59
4.10	Sequence diagram: "Return Handling"	60
4.11	Caliper Finalization Interface	61
4.12	Finished Calipers Inventory - Administrator View	61
4.13	Finished Calipers Inventory - Agent Restricted View	62
4.14	Administrative Feature - Facture (Invoice) Generation	62
4.15	Administrative Overview Dashboard	63
4.16	Delivery Management Interface	63
4.17	Return Handling - Initial Request Form	64
4.18	Return Handling - Serial Number Verification	64
4.19	Return Process Success Confirmation	65
4.20	Caliper in "Returned Insurance" State	66
5.1	Use case "Spare Parts Management"	69
5.2	Use case "AI Piece Recognition"	70
5.3	Use case "Voice Command Entry"	71
5.4	Sequence diagram: "Spare Parts Management"	73
5.5	Sequence diagram: "AI Recognition Logic"	74
5.6	Sequence diagram: "Voice Command Entry Logic"	75
5.7	Spare Part Assignment Form	76
5.8	Spare Part History Overview	76
5.9	New Spare Part Registration	77
5.10	Registration Form - Error Handling	77
5.11	AI Recognition - Camera Access	78
5.12	AI Recognition - Result Confirmation	78
5.13	Voice Command - Active Recording Interface	79
5.14	Voice Command - Query Success Message	80
5.15	Voice Command - Error Handling Interface	80

List of Tables

1.1	Comparative Study of Existing Solutions	16
1.2	Backend Technologies	18
1.3	Frontend Technologies	19
1.4	Programming Language	19
1.5	Overview of MongoDB	20
2.1	Specification of Requirements	25
3.1	Sprint Backlog: Sprint 1	31
3.2	Use Case Description – User Authentication	33
3.3	Use Case Description – Account Management	34
3.4	Use Case Description – Calipers Management	35
3.5	Use Case Description – Data Import Engine	36
4.1	Sprint Backlog: Sprint 2	50
4.2	Use Case Description – Caliper Finalization	51
4.3	Use Case Description – Finished Calipers Management	52
4.4	Use Case Description – Calipers Overview	53
4.5	Use Case Description – Calipers Delivery Management	54
4.6	Use Case Description – Return Handling	55
5.1	Sprint Backlog: Sprint 3	68
5.2	Use Case Description – Spare Parts Management	70
5.3	Use Case Description – AI Piece Recognition	71
5.4	Use Case Description – Voice Command Entry	72

List of abbreviations

Terme	Description	Pages
A		
AI	Artificial Intelligence.	22,23,24,27,65,67
API	Application Programming Interface.	19
C		
CMMS	Computerized Maintenance Management System.	15
CSS	Cascading Style Sheets.	18
D		
DOM	Document Object Model.	18
G		
GUI	Graphical User Interface.	39, 60, 74
H		
HTML	HyperText Markup Language.	18
HTTP	HyperText Transfer Protocol.	19
I		

LIST OF TABLES

Terme	Description	Pages
ISITCOM	Institut Supérieur d'Informatique et des Technologies de Communication.	13
J		
JSON	JavaScript Object Notation.	19
JSX	JavaScript XML.	18
JWT	JSON Web Token.	24
M		
MERN	MongoDB, Express, React, Node.js.	83
MVC	Model-View-Controller.	28
R		
REST	Representational State Transfer.	19
U		
UI	User Interface.	15
UML	Unified Modeling Language.	20, 31, 49, 68
V		
VIN	Vehicle Identification Number.	15

General Introduction

The management and maintenance of bus engines represent a major challenge in both public and private transportation sectors. Ensuring vehicle reliability, reducing breakdowns, and optimizing maintenance operations are essential to guarantee safety, service continuity, and cost efficiency. In this context, digital solutions play a key role in modernizing traditional management processes.

This end-of-studies project focuses on the development of two complementary software solutions: a web application and a mobile application dedicated to bus engine management. The proposed system aims to facilitate the monitoring of bus engines, maintenance operations, and technical data through an intuitive and centralized digital platform adapted to non-technical users.

The motivation behind this project stems from the limitations of traditional bus maintenance management methods, which often rely on manual processes, paper-based records, or fragmented tools. These approaches can lead to data inconsistency, delayed maintenance, and increased operational costs. The proposed solution seeks to overcome these limitations by providing a practical, efficient, and user-friendly system.

In this report, we will present an analysis of existing solutions, identify the challenges related to conventional bus engine management, and propose an innovative digital solution to address these issues. We will also detail the functional specifications, system architecture, and implementation of the developed web and mobile applications in the following chapters.

CHAPTER 1

General Framework of the Project

Chapter Contents

Introduction	14
1.1 Project Presentation	14
1.2 Presentation of the Host Organization	14
1.3 Study of the Existing Situation	15
1.4 Project Management Methodology	17
1.5 Technical Selections	18
1.6 Conclusion	21

Chapter 1

General Framework of the Project

Introduction

In order to successfully carry out the design and development phases, a preliminary study must be conducted. First, we focus on the general framework of the project and the host organization. Then, we present existing solutions while identifying the related problem in order to define the objectives of our work. Finally, we describe the adopted methodology and the specification of requirements.

1.1 Project Presentation

This project entitled “*Development of Web and Mobile Applications for Bus Engine Management*” was proposed by the company Bus Software as part of a final-year project, in agreement with the Higher Institute of Computer Science and Communication Technologies (ISITCOM Sousse).

1.2 Presentation of the Host Organization

1.2.1 Presentation of the Company

Bus Software is a software development company founded in April 2016. It specializes in intelligent transportation systems and provides mobility assistance solutions for urban and intercity public transportation.

The main project of Bus Software consists of the design, integration, operation, and maintenance of ticketing and multimodal information systems.



Figure 1.1: Company Logo

1.2.2 Activities and Services

The company provides innovative solutions for managing onboard fare collection systems in buses. These systems interact with various services such as audio announcements, video surveillance, geolocation, and Wi-Fi.

Winicari, the company's mobile application, serves as a personalized travel guide for public transport users in Tunisia. It provides real-time information and step-by-step guidance for efficient travel.

1.3 Study of the Existing Situation

1.3.1 Project Context

The Bus Software project addresses the need for maintenance workshops to adopt digital solutions for managing spare parts, including reception, repair, and delivery. The system is designed as a unified web and mobile platform enabling centralized management.

1.3.2 Problem Statement

Maintenance workshops rely heavily on manual processes such as paper-based records. This leads to delays, errors, and lack of traceability. Additionally, the absence of centralized systems limits visibility and reduces productivity.

1.3.3 Comparative Study

Applications	Strengths	Weaknesses
Simply Fleet	Mobile-first maintenance logging with real-time updates and offline support. AI-assisted data extraction and voice input.	Not optimized for bus motor repair workflows. No AI recognition of parts or serial numbers.
Bus CMMS	Advanced bus fleet analytics and predictive maintenance.	High dependency on sensors and telematics. Limited mobile AI features.
Fleet Maintenance Manager	Enterprise-grade work order tracking and barcode/VIN scanning.	Not optimized for mobile speed. No vision-based recognition.
AUTOsist	Simple interface and low-cost solution for SMEs.	No AI or automation features. Generic fleet usage.
Fleetio	Strong automation and scalable maintenance system.	Rule-based automation only. No AI vision features.
BUS Software	AI-based recognition of bus motor parts via images. Simple UI for technicians.	Requires training dataset. Sensitive to image quality.

Table 1.1: Comparative Study of Existing Solutions

1.3.4 Proposed Solution

Bus Software proposes a centralized smart platform that ensures real-time tracking and secure management of spare parts. It improves transparency, reduces errors, and enhances communication between stakeholders.



Figure 1.2: Application Logo

1.4 Project Management Methodology

1.4.1 Choice of Methodology

Given the evolving nature of the project, Agile methodology was chosen, specifically SCRUM, due to its flexibility and iterative approach.

1.4.2 SCRUM Methodology

SCRUM is an iterative development methodology based on short cycles called sprints (2–4 weeks). It allows frequent delivery of functional increments and continuous improvement.

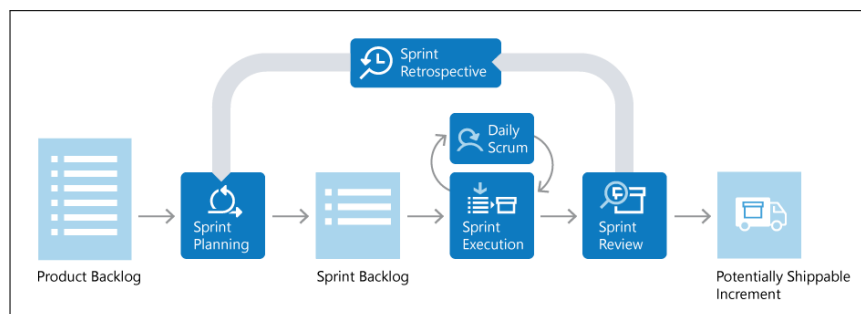


Figure 1.3: SCRUM Lifecycle

1.4.3 SCRUM Principles

- **Team:** Focus on individuals and interactions
- **Application:** Priority to functional software
- **Collaboration:** Continuous client involvement
- **Adaptability:** Acceptance of changes

1.4.4 SCRUM Roles

- Product Owner: Makram Erguez
- Scrum Master: Makram Erguez
- Developer: Mohamed El Ouardi
- Tester: Hichem Ladhari

1.4.5 Advantages of SCRUM

SCRUM allows continuous client feedback, early validation, flexibility, and improved product quality.

1.5 Technical Selections

1.5.1 Backend Technologies

Logo	Description	Functionalities
	Node.js is a server-side JavaScript runtime environment that allows the execution of JavaScript code outside the web browser. It is designed to build scalable network applications.	Non-blocking I/O, Event-driven, npm

Table 1.2: Backend Technologies

1.5.2 Frontend Technologies

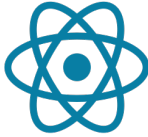
Logo	Description	Functionalities
	React is a JavaScript library for building dynamic user interfaces, promoting efficient rendering and state management through reusable components.	Component-based architecture, Virtual DOM, JSX, Unidirectional data flow
	React Native allows developers to create mobile apps using React and JavaScript, which are then compiled into native platform components.	Cross-platform, Native components, Hot reloading, Performance optimization
	Tailwind CSS is a utility-first CSS framework that enables developers to build custom, responsive designs directly in HTML.	Utility-first, Responsive design, Customizable, PurgeCSS integration

Table 1.3: Frontend Technologies

1.5.3 Programming Language

Logo	Description	Functionalities
	JavaScript is a dynamic programming language commonly used in web development to add interactivity and complex behaviors to websites.	Dynamic typing, Event-driven, Prototypal inheritance, Asynchronous programming

Table 1.4: Programming Language

1.5.4 Database

Logo	Description	Functionalities
	MongoDB is a NoSQL database designed to store data in flexible, document-based formats. It is widely used in modern applications to handle large and evolving datasets. Its structure allows developers to work more easily with JSON-like data.	Flexible schema ,High scalability ,Fast data access, Document-oriented storage

Table 1.5: Overview of MongoDB

1.5.5 API Implementation

REST API

REST stands for Representational State Transfer, a design approach for networked applications. It leverages standard HTTP methods such as GET, POST, PUT, and DELETE to interact with resources, making it straightforward to build and interact with web services. REST APIs are widely adopted for their simplicity and how they allow systems to exchange information efficiently and in a standardized way.

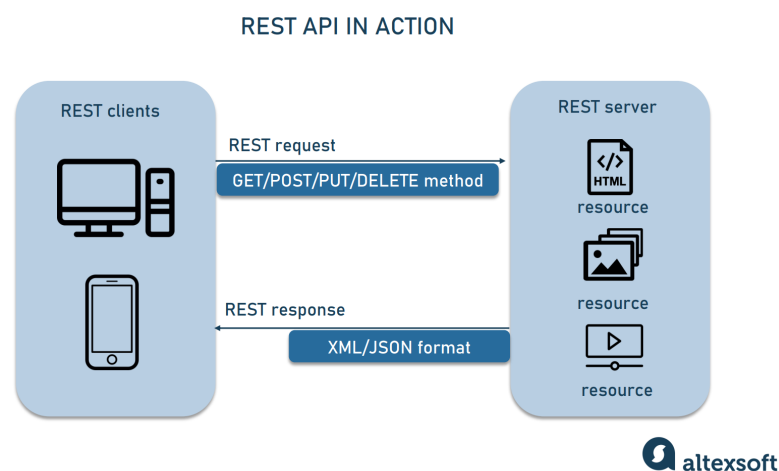


Figure 1.4: RESTful API Architecture and Communication Flow

1.5.6 Gantt Diagram

This illustrates the timeline in which we did each sprint.

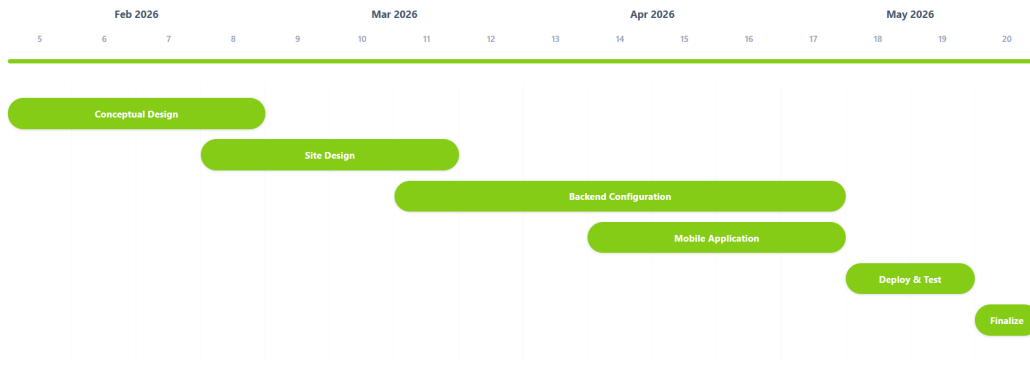


Figure 1.5: Gantt Diagram

1.6 Conclusion

This chapter presented the general framework of the project, the host company, and the study of the existing system. It also described the adopted methodology and technical choices.

In the next chapter, we will detail the product backlog and the functional and non-functional requirements, followed by UML use case diagrams.

CHAPTER 2

Project Initiation of the Project

Chapter Contents

2.1	Introduction	23
2.2	Requirements Capture	23
2.3	Functional Requirements	24
2.4	Specification of Requirements	24
2.5	Semi-Formal Specification	26
2.6	MVC Architecture	29
2.7	Conclusion	29

Chapter 2

Project Initiation

2.1 Introduction

The specification of requirements constitutes the foundation of our project, allowing us to clearly define the problem and the objectives. This will help us establish the initial backlog and create a first sprint planning. We will then address the crucial phase of the Scrum development cycle, namely the “planning” phase, followed by the design phase.

2.2 Requirements Capture

This step is essential and unavoidable, as it allows end users to clearly express their needs and understand the functionalities that the system will offer. In addition, it must take into account the non-functional requirements that will be integrated during the development phase.

2.2.1 Identification of Actors

The application involves the interaction of two main actors:

Administrator: The Administrator is the privileged actor who can manage user accounts, approve or reject return requests, complete return cycles, import reference data (clients, calipers, pieces) from Excel files, restore previous backups, train the AI piece recognition model, and oversee all reception records across the system.

Agent (User): This is the main actor who can create brake caliper reception records, update their processing status, fill in technical details, request a return with a reason, mark items as delivered, and identify spare parts using the AI image recognition feature or the voice command mode.

2.3 Functional Requirements

2.3.1 For Administrator

- **Authenticate:** The administrator must be able to log in to the system with valid credentials securely.
- **User Management:** The administrator must be able to create, modify, delete, and view staff accounts (Agents), controlling their access levels.
- **Dashboard & Monitoring:** The administrator must be able to view a global overview of business activities, including recent receptions, client statistics, and inventory status.
- **Data Management (CRUD):** The administrator must be able to fully manage the core database entities (add, edit, delete Clients, calipers, and Pieces).
- **System Configuration:** The administrator must be able to update application settings and localization preferences.

2.3.2 For Agent

- **Authenticate:** Staff must log in with valid credentials that are encrypted for security.
- **Voice-Activated Reception (Agent Mode):** The agent must be able to record voice commands that the system will automatically transcribe, parse, and translate into database queries to create new reception records.
- **Manual Reception Management:** The agent must be able to manually create, consult, and update reception details.
- **Data Consultation:** The agent must be able to search and view existing Clients and calipers to match them with incoming receptions.
- **Piece Recognition:** The agent must be able to utilize AI features (voice and/or scanning) to identify pieces and automatically file them into the correct reception.

2.4 Specification of Requirements

Below is the table of requirements for the Bus Software platform, categorized by priority, theme, user story, and estimated days for completion.

Table 2.1: Specification of Requirements

Role	Functionality	Description	Priority	Estimation (Days)
Agent	User Authentication	Authenticate using username and password (JWT), invalidate session on logout.	High	1
	Calipers Management	manage the list of brake caliper in reception .	High	5
	Caliper Finalization	Finalize caliper by giving it a serial number	High	3
	Spare Parts Management	manage spare parts related to each caliper	High	5
	AI Piece Recognition	Photograph a spare part to identify it automatically using the AI model.	Low	1
	Voice Command Mode	Record a voice command to create a reception automatically via speech-to-text AI model.	Low	4
Admin	User Authentication	Authenticate using username and password (JWT), invalidate session on logout.	High	1
	Caliper Finalization	Finalize caliper by giving it a serial number	High	3
	Spare Parts Management	manage spare parts related to each caliper	High	5
	AI Piece Recognition	Photograph a spare part to identify it automatically using the AI model.	Low	1
	Voice Command Mode	Record a voice command to create a reception automatically via speech-to-text AI model.	Low	4

Continued on next page

Table 2.1 – *Continued from previous page*

Role	Functionality	Description	Priority	Estimation (Days)
	Data Import Engine	Bulk import clients, brake calipers, or pieces via manual Excel upload or automatic folder synchronization.	High	2
	Account Management	Create, list, and delete agent accounts.	High	1
	Calipers Overview	Access and review monthly caliper logs and agent's bonuses.	Medium	1
	Calipers Delivery Management	Update caliper statuses to "delivered" and manage the final dispatch process.	High	3
	Return Handling	Process and document calipers returned by clients, ensuring inventory and status records are updated.	Low	2

2.5 Semi-Formal Specification

2.5.1 Use Case Diagrams

Use case diagrams give a clear picture of how a system should work, helping guide both the design and development toward solutions that truly fit user requirements. By combining a well-defined plan with a solid understanding of user interactions, they help keep the project on the right track. This approach also ensures that the work follows best practices while staying focused on the needs of the users.

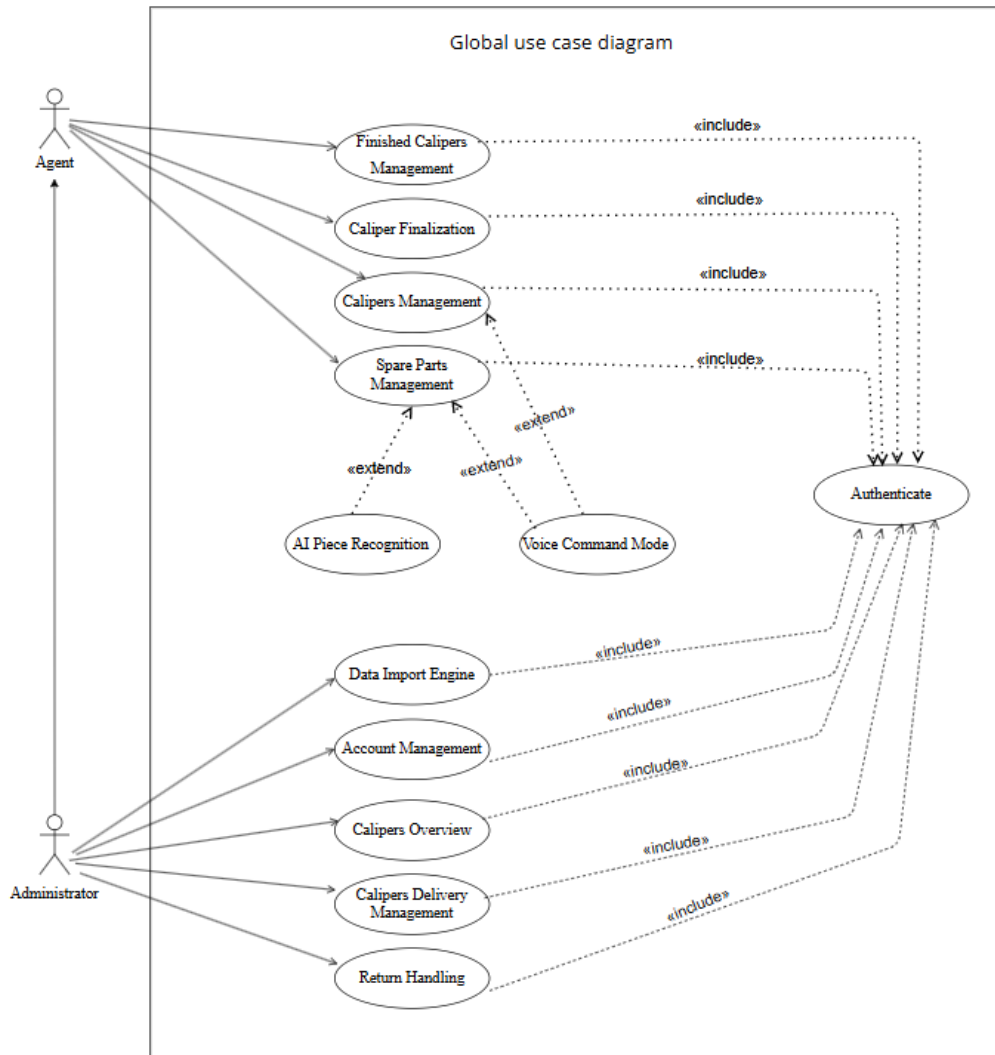


Figure 2.1: Use Case Diagram

2.5.2 Class Diagram

This section presents the class diagram, outlining the system's structure by showing the relationships between classes, their attributes, and methods. The diagram provides a clear overview of how different components interact, ensuring a cohesive and scalable design. It helps visualize the architecture and guides the development process.

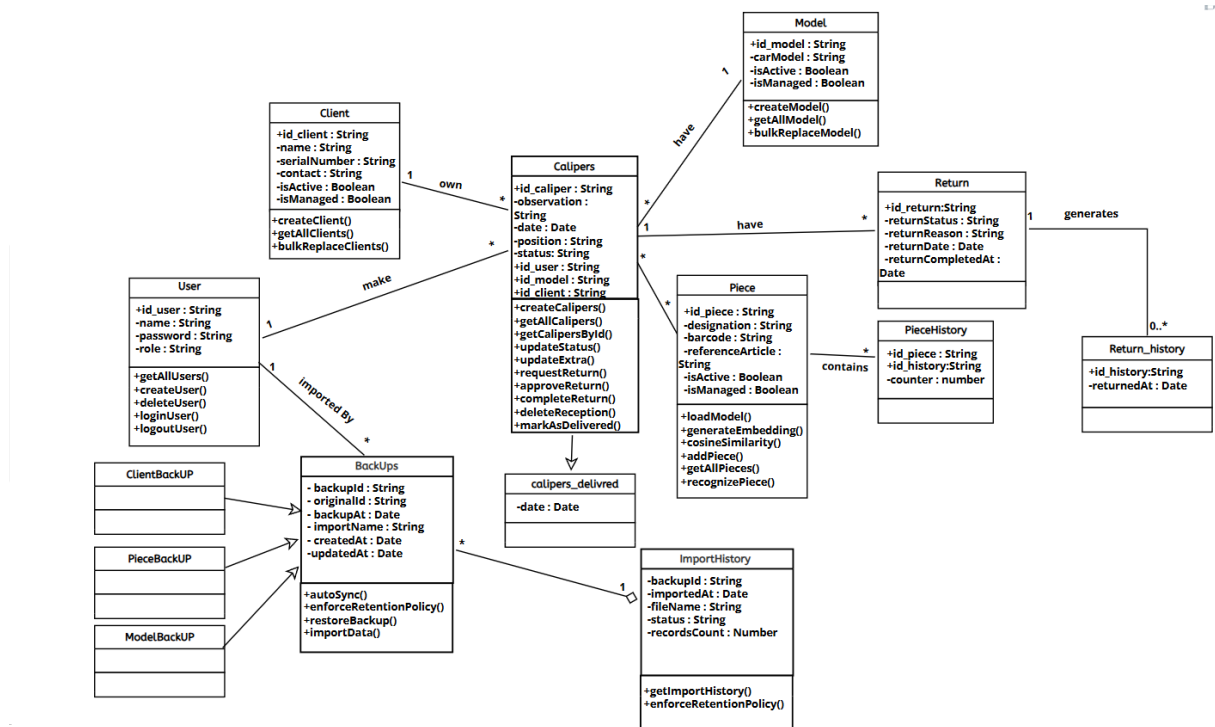


Figure 2.2: Class Diagram

2.5.3 Sprint Planning

This section outlines the project's overall sprint planning, which is divided into three main phases as shown in Figure 2.3. The first sprint, **Sprint 1: Core System Setup**, focuses on establishing the basic foundation and functionalities, including user authentication and data management. Following this, **Sprint 2: Caliper Lifecycle** covers the complete workflow of brake caliper operations, from reception to final dispatch. Finally, **Sprint 3: Parts Management and AI Integration** introduces advanced AI features.



Figure 2.3: Sprint Planning

2.6 MVC Architecture

The MVC architecture stands for Model-View-Controller, it is a commonly used design pattern for organizing the graphical interface of a program, represented by Figure 2.4.

Model: It is where the data is stored and manipulated through the program. It manages access to the data and also offers means for updating these data.

View: The responsibility for displaying data lies here. This mechanism is capable of providing multiple views of the same data, which may be partial or complete.

Controller: It receives commands from the user and modifies data within the model and the view.

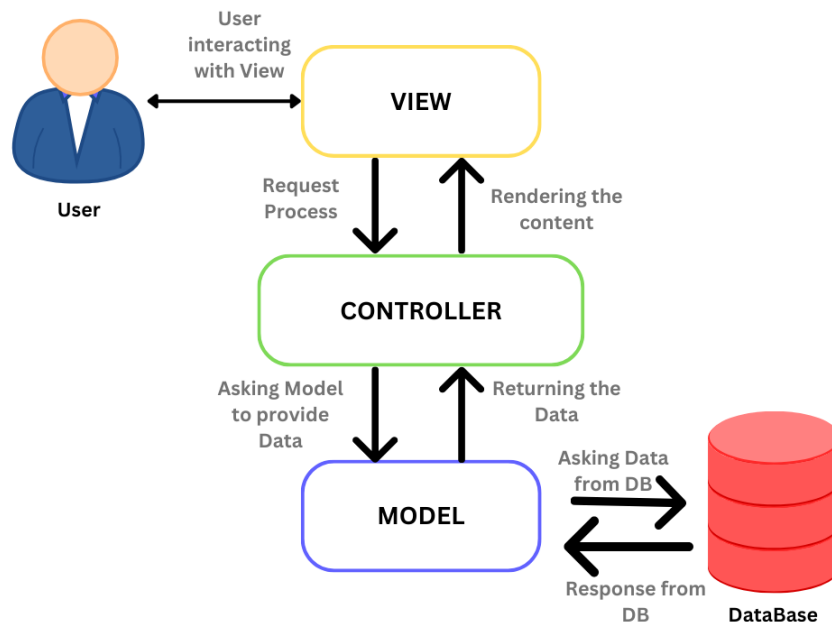


Figure 2.4: Architecture MVC

2.7 Conclusion

In this chapter, we've set up the groundwork by defining what our project needs and how we'll approach building it using Scrum. This is important because it helps us see everything more clearly. As we progress, we will closely examine each part of the project, ensuring that all actions align with our main goals.

CHAPTER 3

Sprint 1: Core System Setup of the Project

Chapter Contents

3.1	Introduction	31
3.2	Sprint Backlog: Sprint 1	31
3.3	UML Diagrams Design	32
3.4	Sequence Diagrams	37
3.5	Interface Design and Prototyping	40
3.6	Conclusion	47

Chapter 3

Sprint 1: Core System Setup

3.1 Introduction

This sprint is dedicated to developing a highly secure login and sign-up interface, which is important in keeping the user data safe while making it easily accessible at the same time. In order to achieve this goal, we will have to design a reliable and efficient process that will feature both password authentication and further security procedures.

3.2 Sprint Backlog: Sprint 1

The Table 3.1 presents the backlog of the sprint 1 describing the functionalities, the user story, and its priority.

Functionality	User Story	Priority
User Authentication	As an agent or an administrator, I can log in securely to access my account.	High
Account Management	As an administrator, I can add or delete users.	High
Calipers Management	As an agent or an administrator, I can manage Calipers relevant to my role.	High
Data Import Engine	As an Admin, I can import model, clients, and pieces data either by uploading an Excel file or by using auto-sync from a server folder, so the catalogue can be populated quickly without manual entry.	Low

Table 3.1: Sprint Backlog: Sprint 1

3.3 UML Diagrams Design

This section highlights the use case and sequence diagrams' design that will be used to explain the functional requirements of the system.

3.3.1 Use Case Diagram: User Authentication

In the diagram below Figure 3.1 and Table 3.2, there is an illustration of how the different users login into a system. Both the Administrator and Agent have to "Authenticate" themselves in order to access the system.

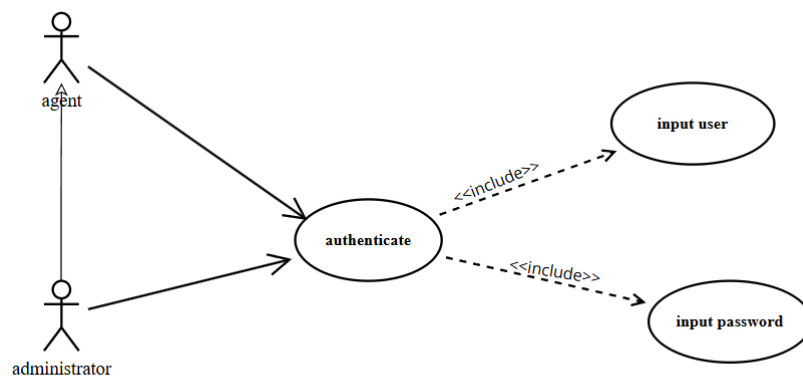


Figure 3.1: Use case "User Authentication"

Use Case	User Authenticate
Actor	Administrator, Agent
Pre-condition	The user must have a registered account with valid credentials
Post-condition	The user is granted access to the system
Main Scenario	1. The user opens the login interface 2. The system displays fields for username and password 3. The user inputs a valid username 4. The user inputs a valid password 5. The system verifies credentials 6. The system grants access to the appropriate interface
Exception	The system displays an error message if the username or password is incorrect

Table 3.2: Use Case Description – User Authentication

3.3.2 Use Case Diagram: Account Management

The following Figure 3.2 and Table 3.3 illustrate how accounts are managed within the system. Only administrators have the authority to manage accounts after proving their identity.

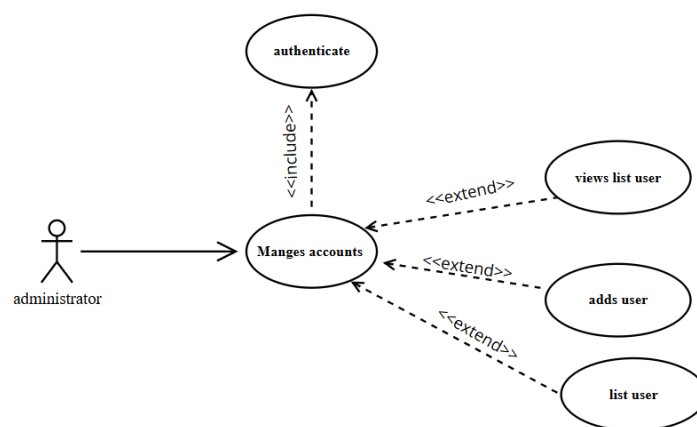


Figure 3.2: Use case "Account Management"

Use Case	Account Management
Actor	Administrator
Pre-condition	The user must authenticate as an admin
Post-condition	The user is granted access to the settings interface
Main Scenario	1. The admin opens the settings interface 2. The system displays the current list of registered users 3. The admin views the users list 4. The admin creates a user 5. The admin deletes a user 6. The system saves changes
Exception	Invalid Data: Error message if required fields are blank or username exists. Self-Deletion: Error message if Admin attempts to delete their own account.

Table 3.3: Use Case Description – Account Management

3.3.3 Use Case Diagram: Calipers Management

Both administrators and agents can manage calipers; however, they must first authenticate their identity with the system.

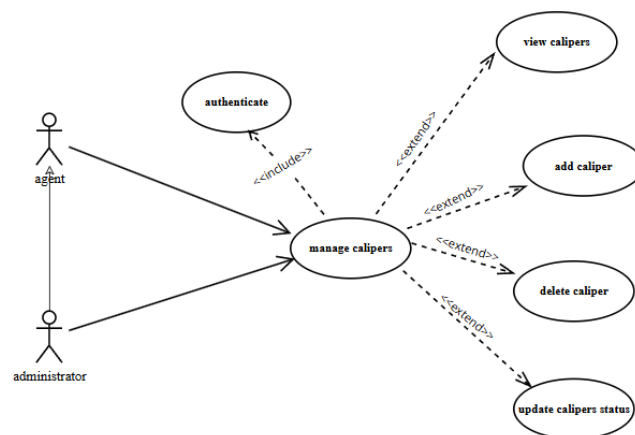


Figure 3.3: Use case "Calipers Management"

Use Case	Calipers Management
Actor	Administrator, Agent
Pre-condition	The user must authenticate
Post-condition	The user is granted access to the reception interface
Main Scenario	1. The user opens the reception interface 2. The system displays the list of calipers 3. The user views the calipers list 4. The user creates a caliper 5. The user edits calipers status 6. The user deletes calipers 7. The system saves changes

Table 3.4: Use Case Description – Calipers Management

3.3.4 Use Case Diagram: Data Import Engine

Only administrators have the authority to manage catalogue data via manual Excel upload or automatic synchronization.

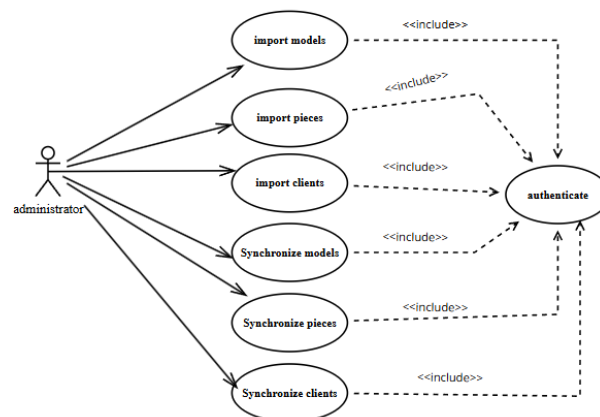


Figure 3.4: Use case "Data Import Engine"

Use Case	Data Import Engine
Actor	Administrator
Pre-condition	Admin authentication required
Post-condition	Access to settings interface granted
Main Scenario	1. Open import settings page 2. System shows: Excel upload / auto-sync 3. Admin selects method 4. Upload file (if Excel) 5. Trigger sync (if auto-sync) 6. System parses & validates data 7. Admin confirms 8. Data saved to catalogue
Exception	Invalid File: Unsupported format Data Failure: Missing/invalid fields Folder Unreachable: Sync folder not accessible

Table 3.5: Use Case Description – Data Import Engine

3.4 Sequence Diagrams

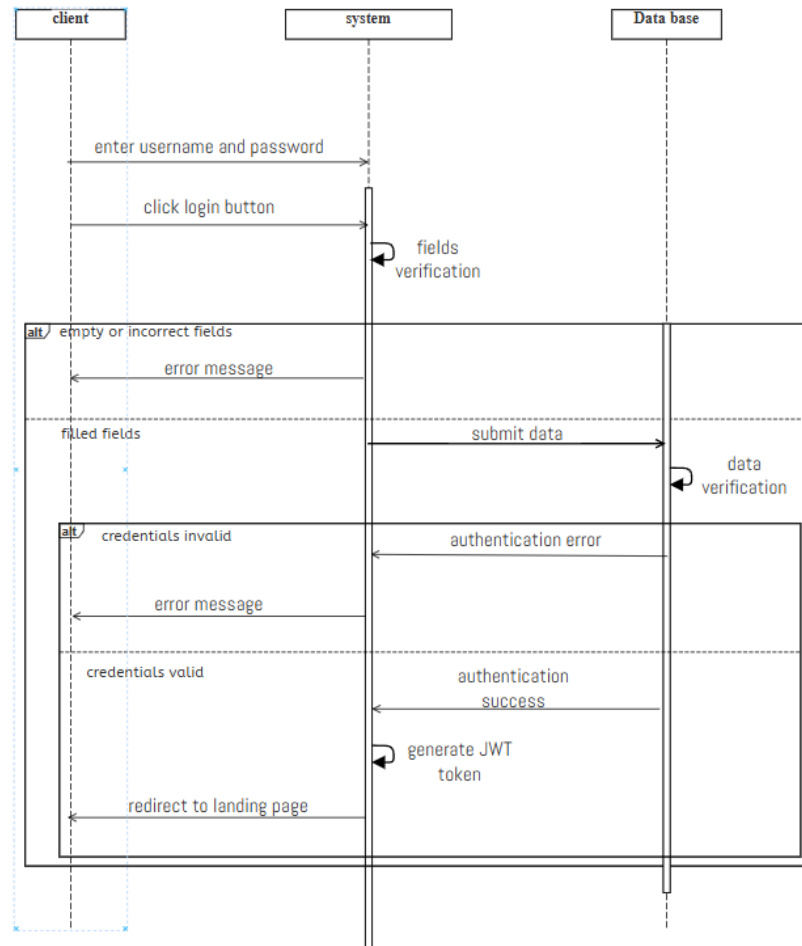


Figure 3.5: Sequence diagram: "User Authentication"

3.4.1 Account Management

Figure 3.6 presents the 'Account Management' sequence diagram, illustrating the account creation process.

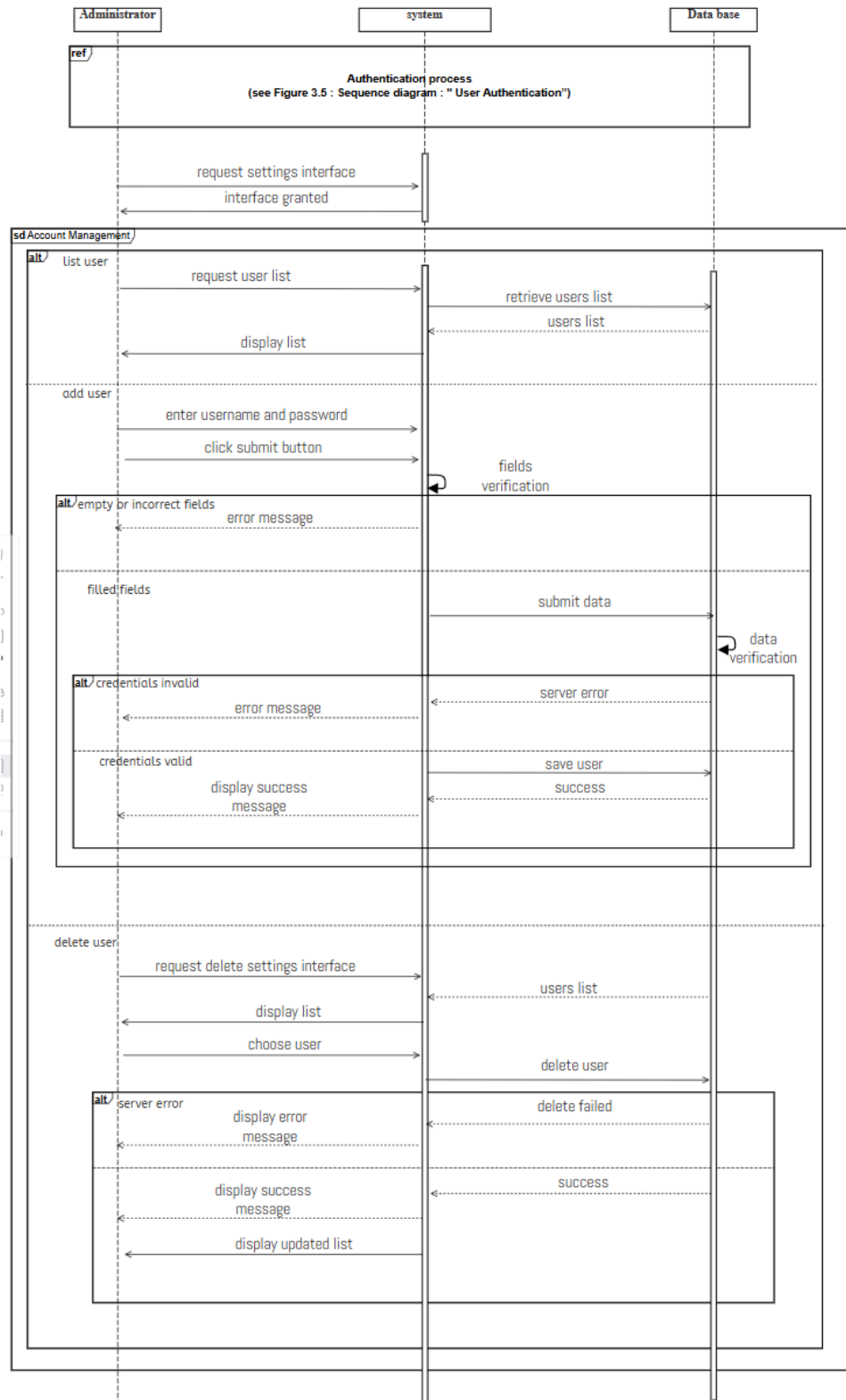


Figure 3.6: Sequence diagram: "Account Management"

3.4.2 Calipers Management

Figure 3.7 illustrates the calipers management process for an authorized user.

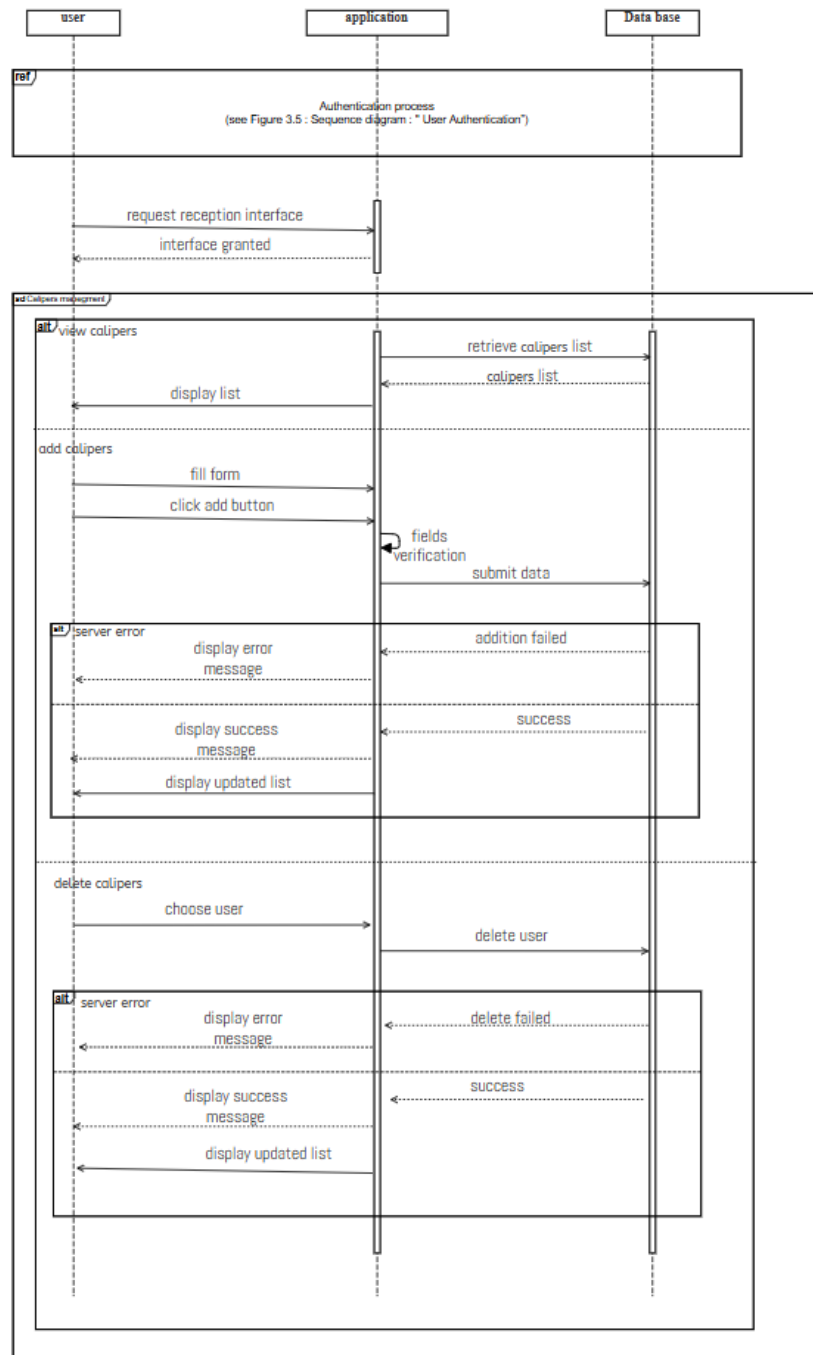


Figure 3.7: Sequence diagram: "Calipers Management"

3.4.3 Data Import Engine

Figure 3.8 presents the sequence diagram for the data import process.

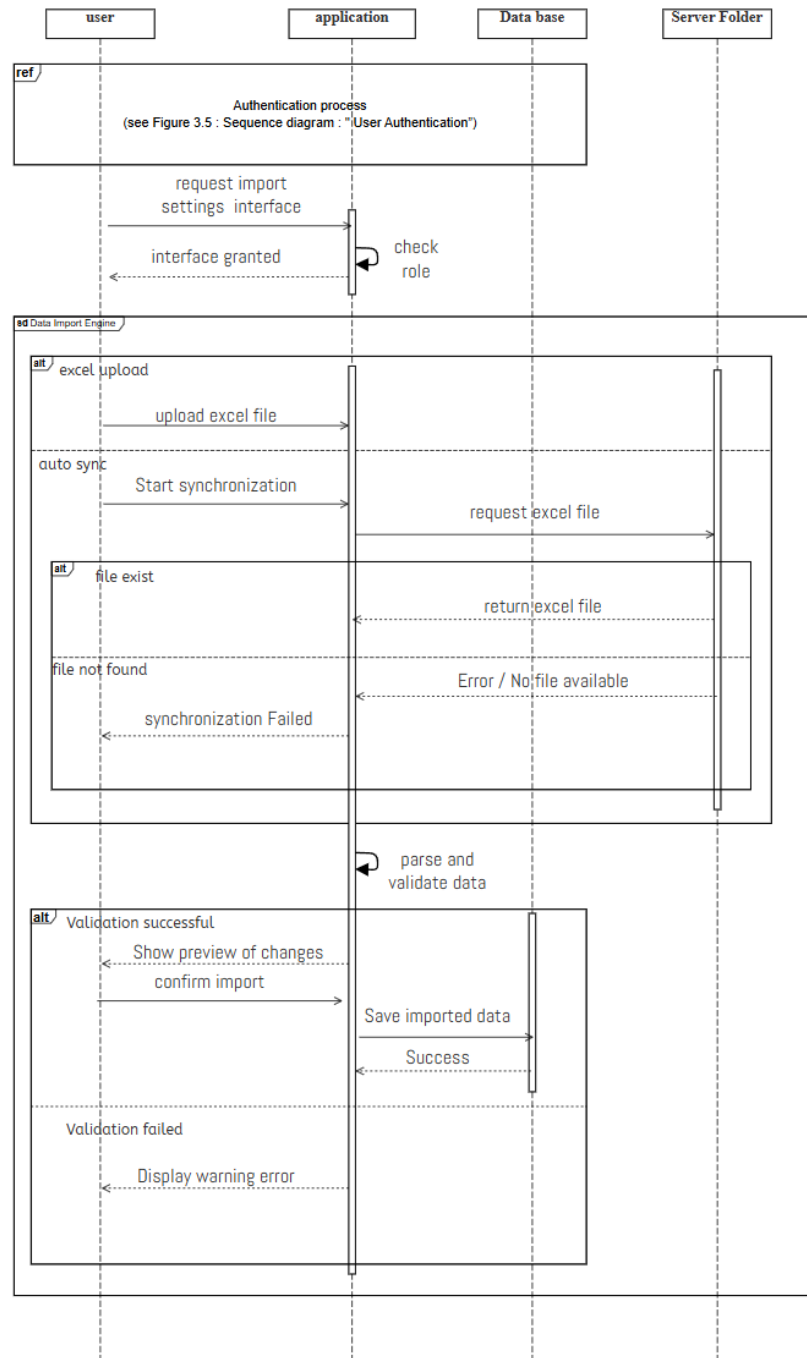


Figure 3.8: Sequence diagram: "Data Import Engine"

3.5 Interface Design and Prototyping

In this section, the GUI design will be discussed for the key functions implemented during this sprint. The screenshots below highlight the GUI of the proposed system.

3.5.1 User Authentication Interfaces

Figure 3.9 presents the interface that displays the initial state of the login portal where users enter their credentials.

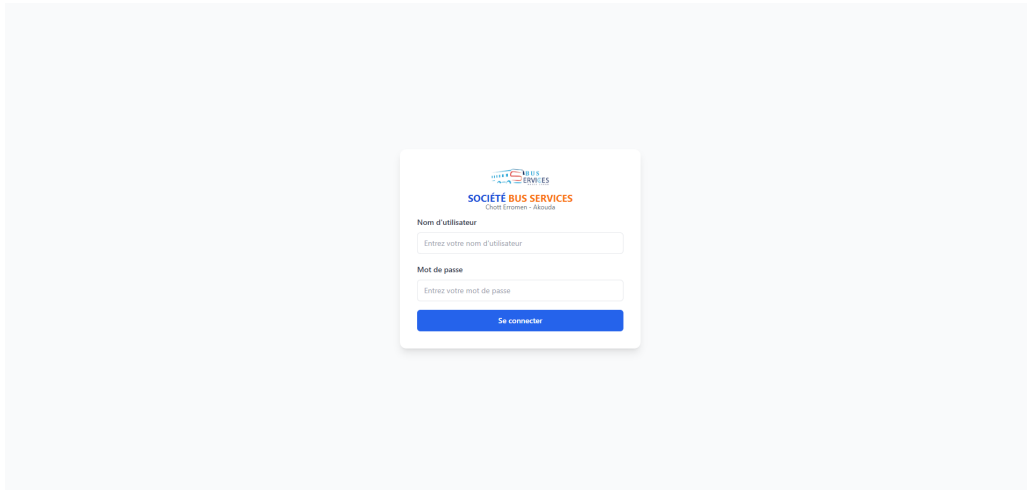


Figure 3.9: Login Interface - Initial State

Figure 3.10 presents the scenario of inputting a wrong username or password, triggering a validation error.

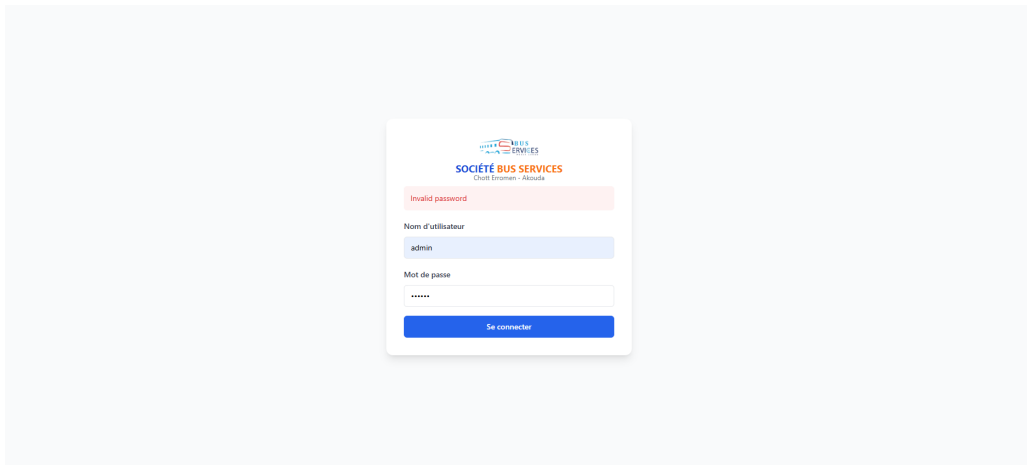


Figure 3.10: Login Interface - Error Handling

3.5.2 Account Management Interfaces

1. The Add Form:

Below is the interface that contains the administrative form through which the information needed to create a new user account will be entered.

The image shows a web application interface with a sidebar on the left and a main content area. The sidebar, titled 'Panneau d'administration', contains links for 'Ajouter un utilisateur', 'Supprimer un utilisateur', 'Mon Compte', and 'Importer des données'. At the bottom of the sidebar are links for 'Accueil' and 'Se déconnecter'. The main content area displays the 'Ajouter un utilisateur' form. This form includes fields for 'Nom d'utilisateur' (with a placeholder 'Nom'), 'Mot de passe' (with a placeholder 'Mot de passe'), and a 'Rôle' dropdown menu currently set to 'Utilisateur'. A blue button labeled 'Ajouter l'utilisateur' is at the bottom of the form.

Figure 3.11: Account Creation Interface - Add Form

2. The User List:

After the creation of the users, the administrator will be able to navigate the user directory view interface where he will be able to browse all records from the database and choose the ones to delete.

The image shows the 'Supprimer définitivement' modal. At the top, it has a title 'Supprimer définitivement' and a warning message: 'Attention : La suppression définitive est irréversible. Toutes les données seront perdues.' Below this is a search bar labeled 'Rechercher un utilisateur' with a placeholder 'Nom...'. The modal displays a list of three users: 'user', 'mohamed', and 'admin2'. Each user entry consists of a red circular icon with a white person silhouette, the user's name, and a red button labeled 'Supprimer'.

Figure 3.12: User Directory View - Selection for Deletion

3. The Delete Modal:

For the user to be deleted successfully, the administrator will need to confirm the action on a confirmation modal.

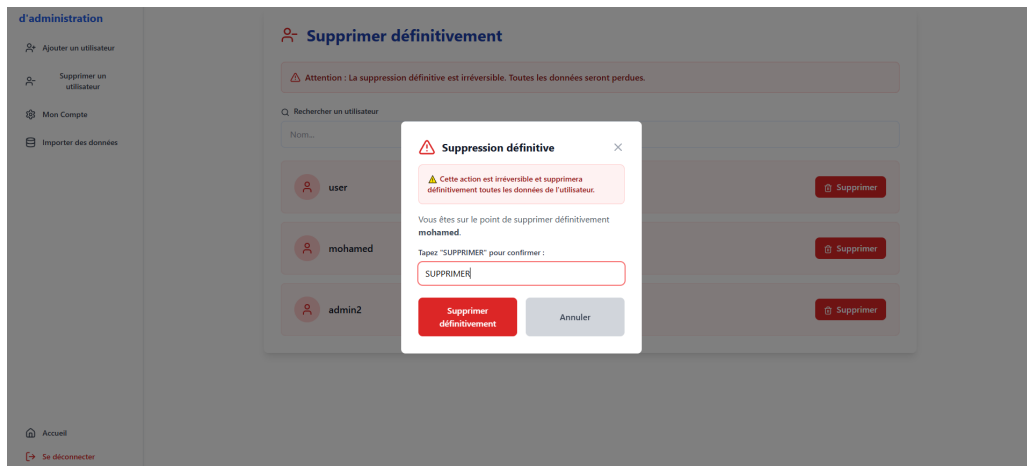


Figure 3.13: Deletion Process - Confirmation Modal

3.5.3 Calipers Management Interfaces

1. Add Caliper Form:

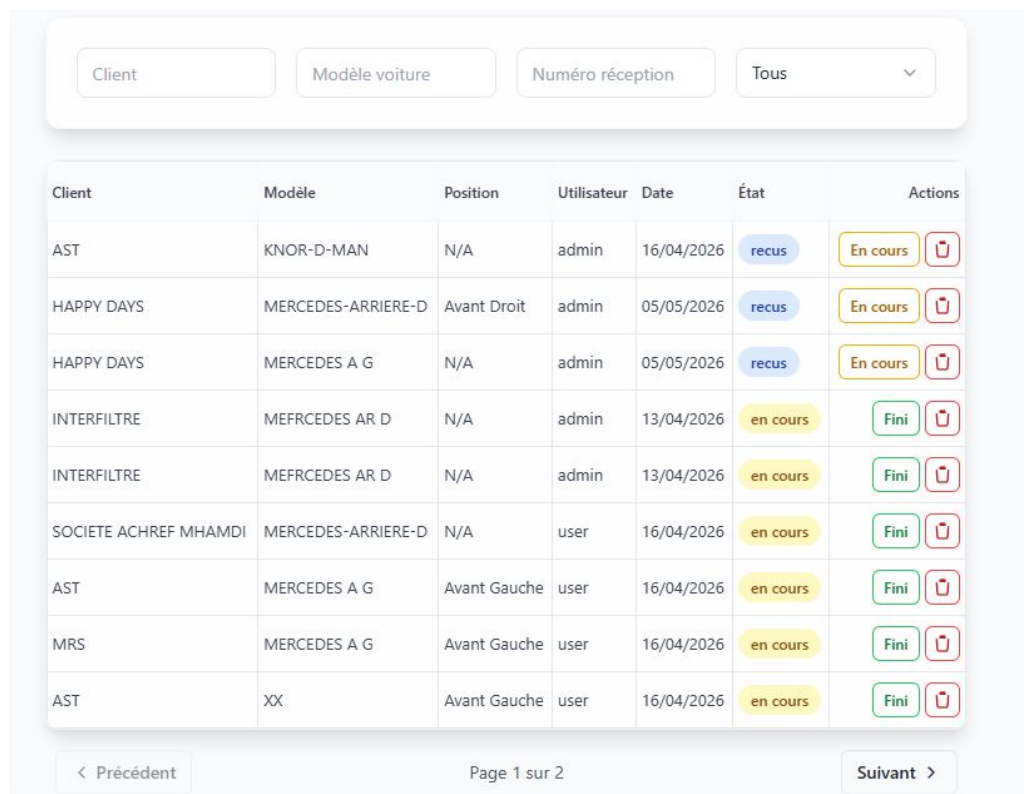
The interface below provides a form used to register a new incoming caliper into the system, capturing essential details such as the model and client information.

The image shows a form titled 'Nouveau Étrier'. It contains several input fields: a required field '*Client' with a sub-label 'NOM DU CLIENT' and a '+' icon; a required field '*Modèle' with a sub-label 'TYPE DE MODÈLE' and a '+' icon; a text area for 'Observation'; a date field for 'Date de réception' showing '04/22/2026' with a calendar icon; and a dropdown menu for 'État' currently showing 'Reçu'. At the bottom is a large blue button labeled 'Enregistrer le produit'.

Figure 3.14: Caliper Management - Add Form

2. Caliper Inventory:

Authorized users are allowed access to this function can view the inventory to review available calipers. It acts as the gateway through which individual calipers can be selected for editing or deletion.



Client	Modèle	Position	Utilisateur	Date	État	Actions
AST	KNOR-D-MAN	N/A	admin	16/04/2026	recus	En cours
HAPPY DAYS	MERCEDES-ARRIERE-D	Avant Droit	admin	05/05/2026	recus	En cours
HAPPY DAYS	MERCEDES A G	N/A	admin	05/05/2026	recus	En cours
INTERFILTRE	MEFRCEDES AR D	N/A	admin	13/04/2026	en cours	Fin
INTERFILTRE	MEFRCEDES AR D	N/A	admin	13/04/2026	en cours	Fin
SOCIETE ACHREF MHAMDI	MERCEDES-ARRIERE-D	N/A	user	16/04/2026	en cours	Fin
AST	MERCEDES A G	Avant Gauche	user	16/04/2026	en cours	Fin
MRS	MERCEDES A G	Avant Gauche	user	16/04/2026	en cours	Fin
AST	XX	Avant Gauche	user	16/04/2026	en cours	Fin

Figure 3.15: Caliper Inventory Overview - Selection Interface

3. Edit Status:

In this section, users have the ability to change the operating status of a caliper depending on where it is in the workflow.

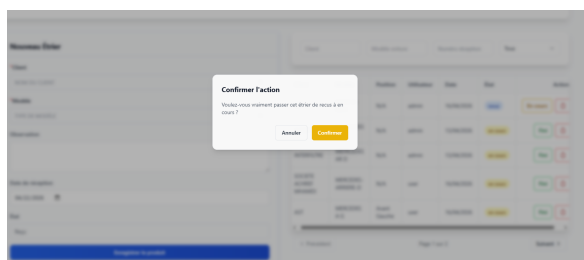


Figure 3.16: Caliper Status Update Step 1

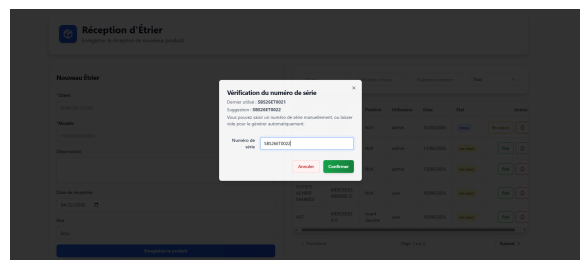


Figure 3.17: Caliper Status Update Step 2

4. Delete Modal:

The deletion process will only be completed after an initial confirmation is done through the deletion modal.

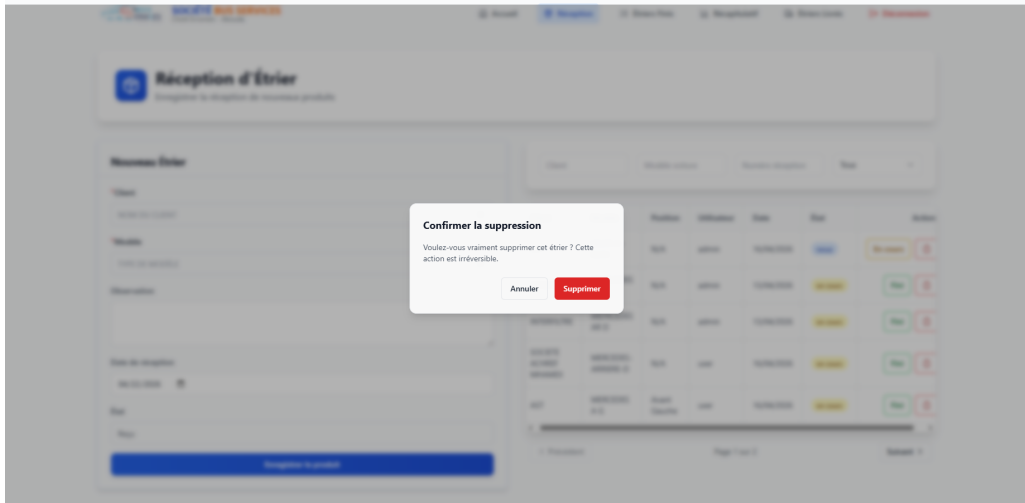


Figure 3.18: Caliper Management - Deletion Confirmation Modal

3.5.4 Data Import Engine Interfaces

The import settings provide administrators with two distinct methods for populating the catalogue: manual Excel file uploads and automated server synchronization.

1. Importation Initial Phase:

Below is the initial data import interface, which is exclusively accessible to authorized administrators to begin the manual file upload process.

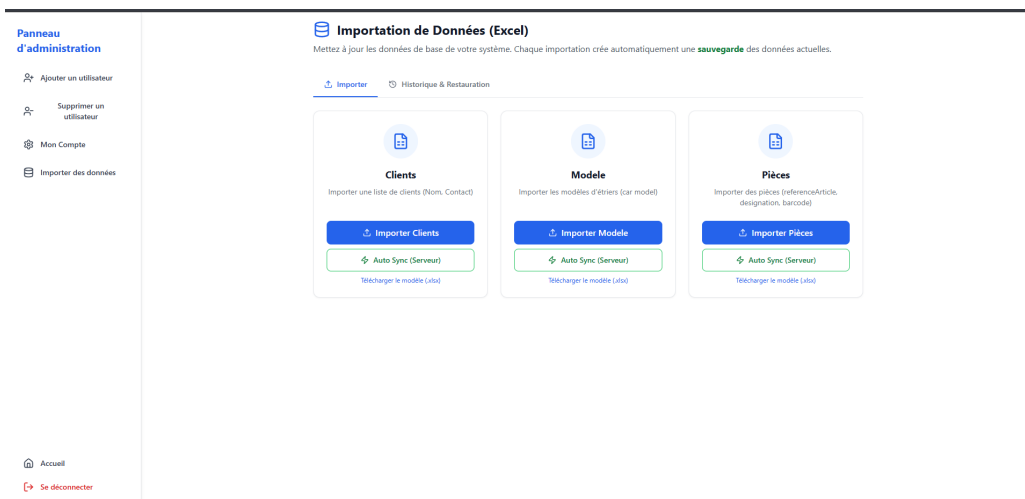


Figure 3.19: Data Import - Initial Phase

2. Preview Interface:

After uploading a file, the administrator is presented with a preview interface to verify the parsed data changes before securely confirming the update to the database.

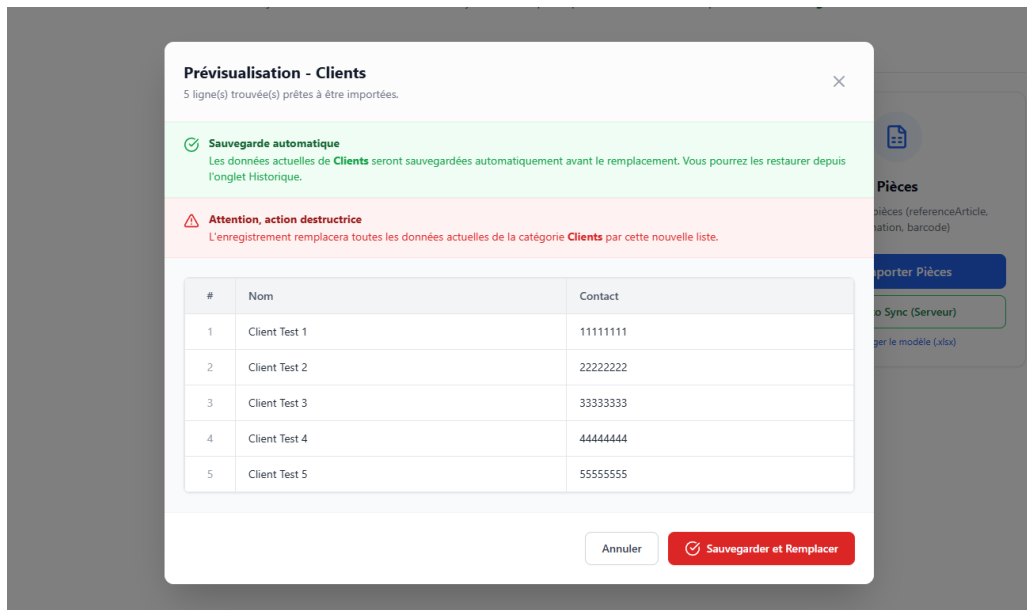


Figure 3.20: Data Import - Preview Interface

3. Errors in Importation:

If the uploaded Excel file is improperly formatted, does not match the required template, or cannot be found, the system halts the process and displays detailed error alerts.

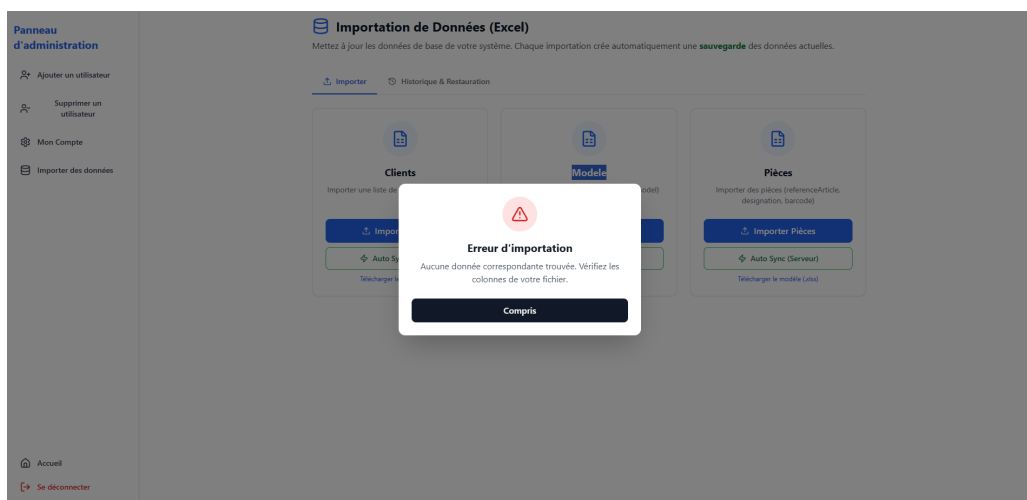


Figure 3.21: Data Import - Error Handling

4. Synchronization Modal:

For automated data population, clicking the auto-sync button triggers a confirmation modal to ensure the administrator intentionally initiates the server synchronization process.

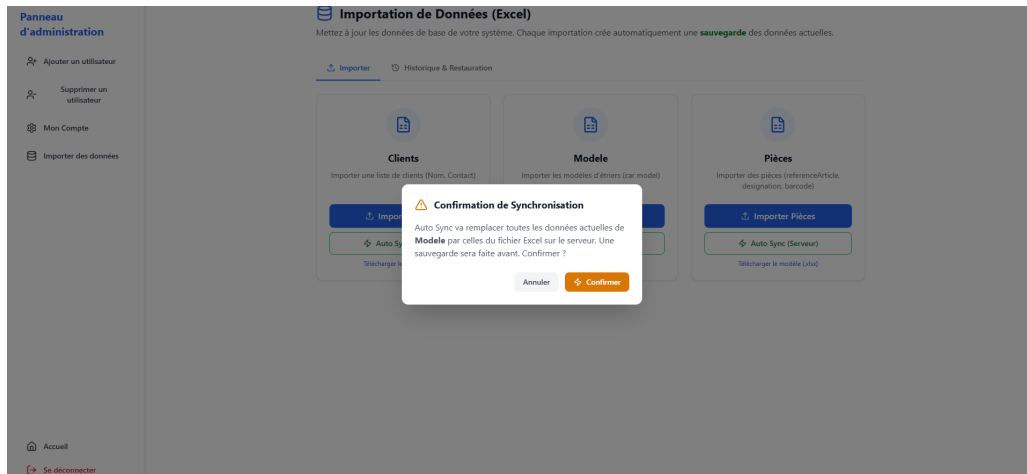


Figure 3.22: Auto-Sync - Confirmation Modal

3.6 Conclusion

To sum up, the sprint was successful in creating the basic structure and functionalities of the software. The tasks included the development of user authentication, account management systems, and the initial calipers reception module. Furthermore, the installation of the Data Import Engine provides an efficient method for filling the catalogue by both manually uploading data from Excel files or synchronizing data on the server side. Being equipped with this secure structure, the program can now move on to solving more complicated workflows within the product life cycle, which will be addressed later in the next chapter **Sprint 2: Caliper Lifecycle**.

CHAPTER 4

Sprint 2: Caliper Lifecycle

Chapter Contents

4.1	Introduction	49
4.2	Sprint Backlog: Sprint 2	49
4.3	UML Diagrams Design	50
4.4	Sequence Diagrams	55
4.5	Interface Design and Prototyping	61
4.6	Conclusion	66

Chapter 4

Sprint 2: Caliper Lifecycle

4.1 Introduction

This sprint is dedicated to managing the complete lifecycle of calipers within the system, which is important in keeping the workflow structured while ensuring smooth delivery and returns at the same time. In order to achieve this goal, we will have to design a reliable and efficient process that will feature both intuitive interfaces and improved data visibility throughout each stage of the caliper lifecycle.

4.2 Sprint Backlog: Sprint 2

The Table 4.1 presents the backlog of the sprint 2 describing the functionalities, the user story, and its priority.

Functionality	User Story	Priority
Caliper Finalization	As an agent or an administrator, I want to mark a caliper as finished so that it can move to the next stage of the workflow.	High
Finished Calipers Management	As an agent or an administrator, I want to access the finished calipers inventory so that I can manage them according to my role's permissions.	High
Calipers Overview	As an administrator, I want to view a recap of all calipers so that I can monitor my workers' workflow and reward or penalize them accordingly.	Medium
Calipers Delivery Management	I want to manage the delivery of finished calipers so that I can ensure they are correctly delivered to our clients.	High
Return Handling	As an agent or an administrator, I want to handle returned calipers so that I can properly manage the return cycle.	High

Table 4.1: Sprint Backlog: Sprint 2

4.3 UML Diagrams Design

This section highlights the use case and sequence diagrams' design that will be used to explain the functional requirements of the system.

4.3.1 Use Case Diagram: Caliper Finalization

The following Figure 4.1 and Table 4.2 illustrate how authorized users finalize a caliper's workflow. Both administrators and agents can mark a caliper as finished, giving it a final serial number and signaling it is ready to be delivered.

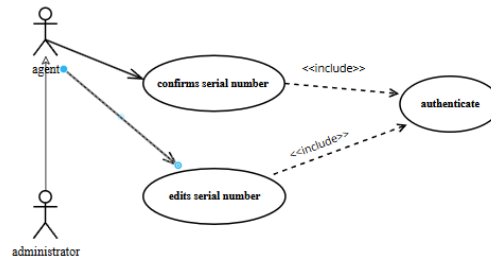


Figure 4.1: Use case "Caliper Finalization"

Use Case	Caliper Finalization
Actor	Administrator, Agent
Pre-condition	The user must be authenticated. The caliper must be in the 'in progress' state.
Post-condition	The caliper's status is updated to 'finished' in the system.
Main Scenario	1. The user opens the reception interface 2. The system displays active calipers 3. The user selects a specific caliper 4. The user clicks the 'Finalize' button 5. The user confirms or edits the final serial number 6. The system saves the changes 7. The caliper is moved to the finished interface

Table 4.2: Use Case Description – Caliper Finalization

4.3.2 Use Case Diagram: Finished Calipers Management

Figure 4.2 and Table 4.3 describe how finished calipers are managed, with role-based access for administrators and agents.

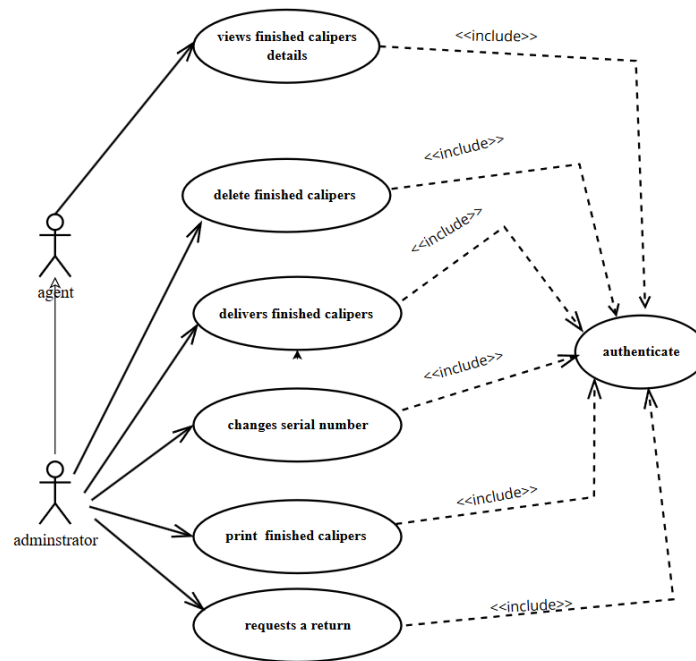


Figure 4.2: Use case "Finished Calipers Management"

Use Case	Finished Calipers Management
Actor	Administrator, Agent
Pre-condition	User authenticated; caliper status = finished
Post-condition	Caliper reviewed/processed based on role
Main Scenario	1. Open "Finished Calipers" interface 2. System displays finished inventory 3. Role-based options are shown 4. User selects a caliper 5. User performs allowed actions 6. System saves changes

Table 4.3: Use Case Description – Finished Calipers Management

4.3.3 Use Case Diagram: Calipers Overview

Figure 4.3 and Table 4.4 present the overview process. This is an administrative function used to monitor worker performance and review the completed workflow.

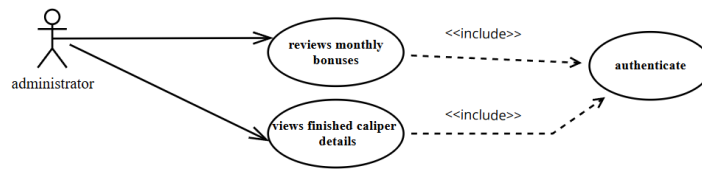


Figure 4.3: Use case "Calipers Overview"

Use Case	Calipers Overview
Actor	Administrator
Pre-condition	The user must authenticate as an admin.
Post-condition	The user views a generated summary of completed tasks and calculated bonuses.
Main Scenario	1. The administrator navigates to the calipers overview dashboard 2. The system fetches the historical records of finished and returned calipers 3. The system calculates and displays global metrics, including the total finished count and the total accumulated bonuses/penalties 4. The system displays a detailed list of calipers grouped by month 5. The administrator expands a specific month to review individual caliper statuses and their impact on the monthly bonus

Table 4.4: Use Case Description – Calipers Overview

4.3.4 Use Case Diagram: Calipers Delivery Management

Figure 4.4 and Table 4.5 illustrate the delivery management process, ensuring that finalized calipers are properly dispatched and recorded as delivered to the clients.

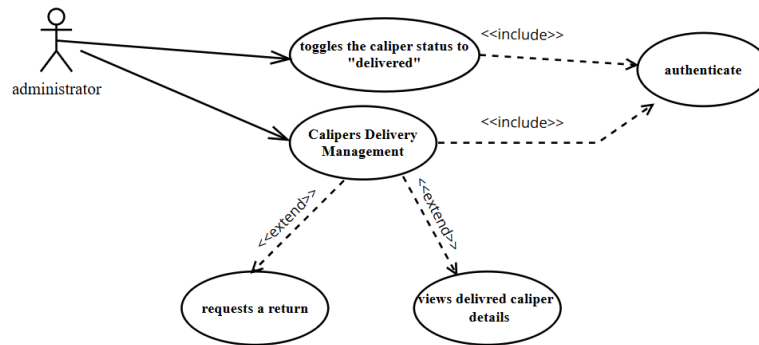


Figure 4.4: Use case "Calipers Delivery Management"

Use Case	Calipers Delivery Management
Actor	Administrator
Pre-condition	The user must be authenticated. The caliper must be marked as 'finished'.
Post-condition	The caliper's status is updated to 'delivered'.
Main Scenario	1. The user navigates to the Finished Calipers interface 2. The system displays a list of ready-to-ship calipers 3. The user selects a caliper for delivery 4. The user clicks on the delivery button 5. The system updates the status to 'delivered' 5. The system pushes the caliper to delivery management interface

Table 4.5: Use Case Description – Calipers Delivery Management

4.3.5 Use Case Diagram: Return Handling

The final stage of the lifecycle involves managing returns. Figure 4.5 and Table 4.6 outline how authorized users process calipers sent back by clients.

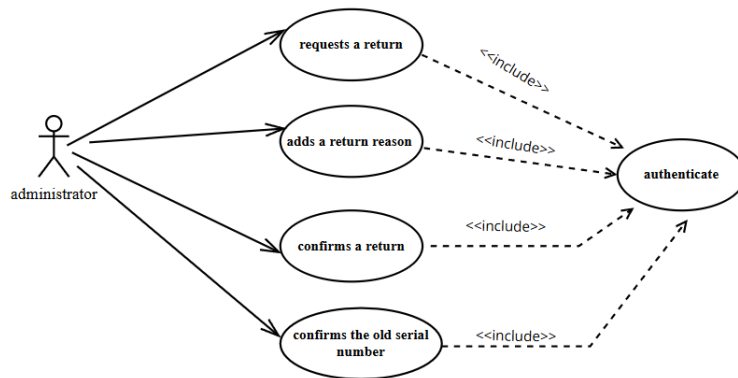


Figure 4.5: Use case "Return Handling"

Use Case	Return Handling
Actor	Administrator ,Agent
Pre-condition	The user must be authenticated. The caliper must currently be finished or returned.
Post-condition	The caliper is registered as 'returned' and placed back into the system workflow.
Main Scenario	1. The user opens the Finished Calipers or Delivery interface 2. The user searches for the specific caliper by serial number or client 3. The user requests a return 4. The user logs the reason for return 5. The user confirms the return 6. The system updates the status to 'returned' 7. The system push the caliper into the workflow (reception) 8. The system saves the changes

Table 4.6: Use Case Description – Return Handling

4.4 Sequence Diagrams

4.4.1 Caliper Finalization

Figure 4.6 presents the sequence diagram illustrating the interactions required to finalize a caliper .

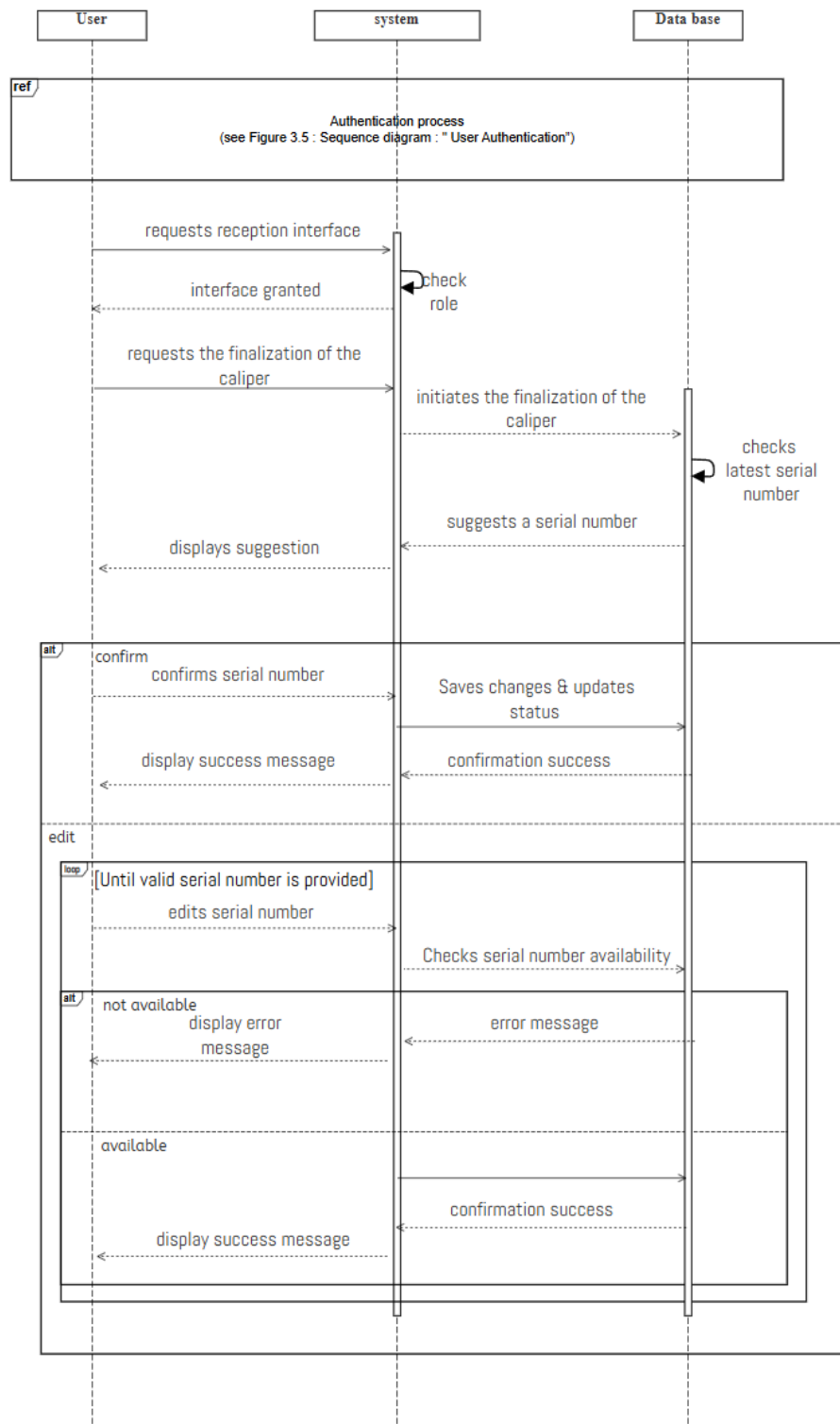


Figure 4.6: Sequence diagram: "Caliper Finalization"

4.4.2 Finished Calipers Management

Figure 4.7 presents the sequence diagram illustrating how the system routes a finalized caliper into the finished inventory and handles role-based permission checks when a user attempts to manage it.

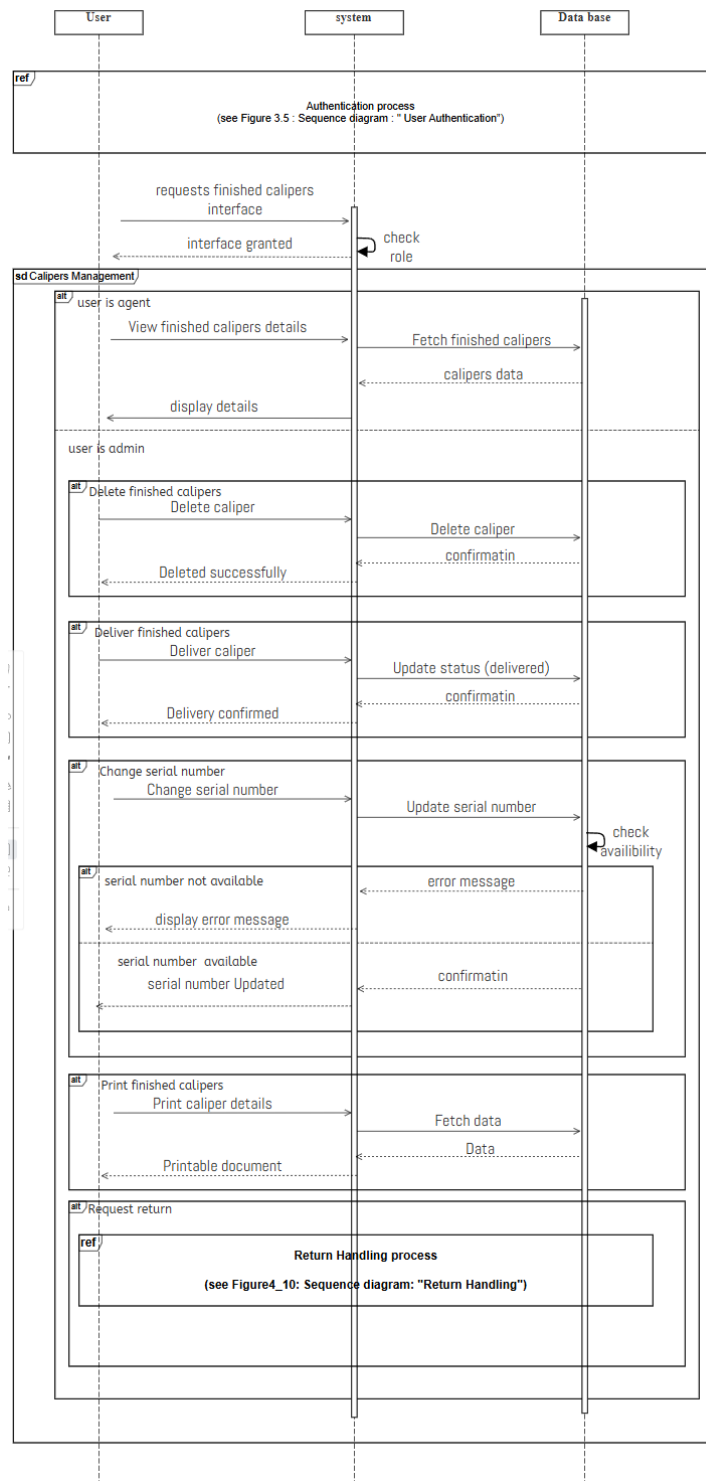


Figure 4.7: Sequence diagram: "Finished Calipers Management"

4.4.3 Calipers Overview

Figure 4.8 presents the sequence diagram illustrating how the system fetches and displays the workflow recap for administrators.

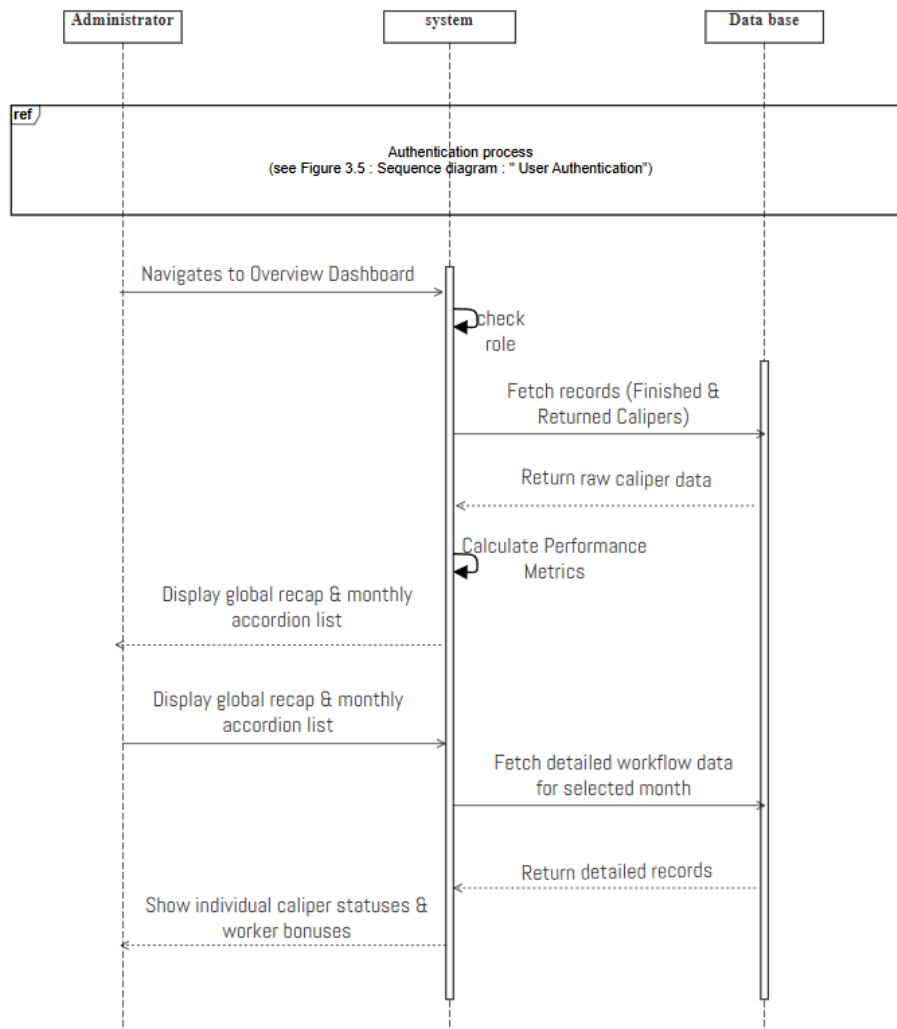


Figure 4.8: Sequence diagram: "Calipers Overview"

4.4.4 Calipers Delivery Management

Figure 4.9 outlines the step-by-step interactions involved in successfully registering a caliper's delivery to a client.

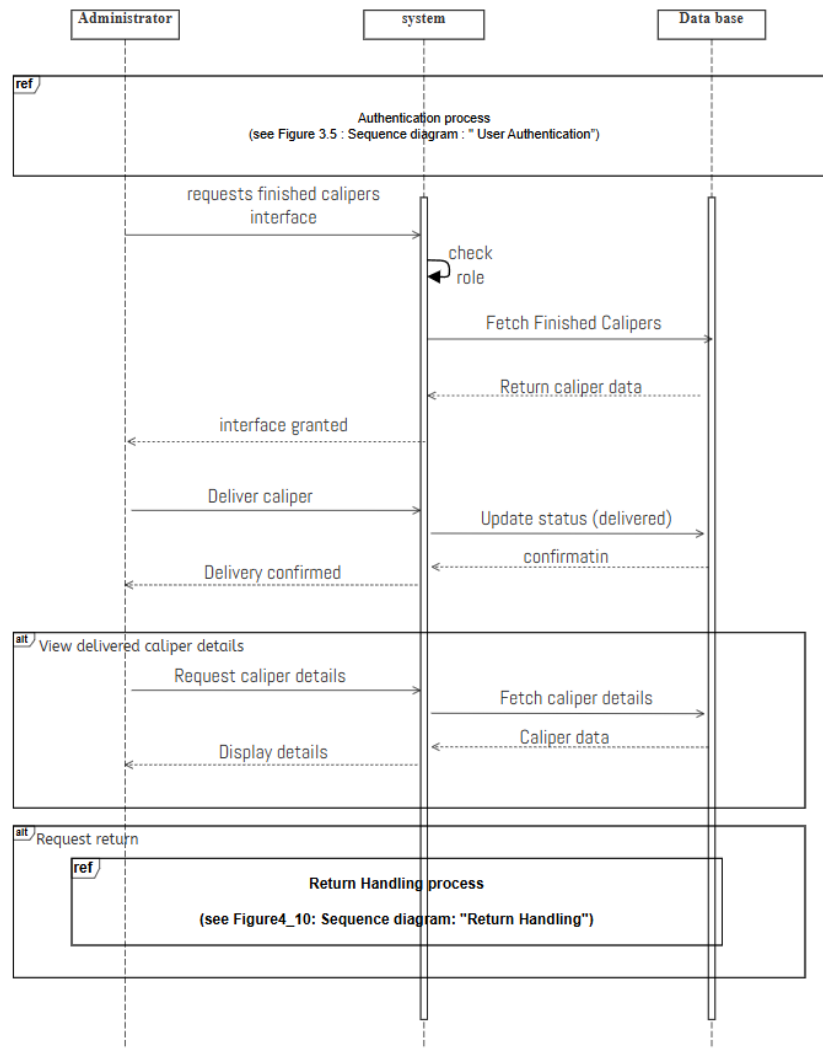


Figure 4.9: Sequence diagram: "Calipers Delivery Management"

4.4.5 Return Handling

Figure 4.10 illustrates the sequence of events triggered when a user logs a returned caliper back into the system.

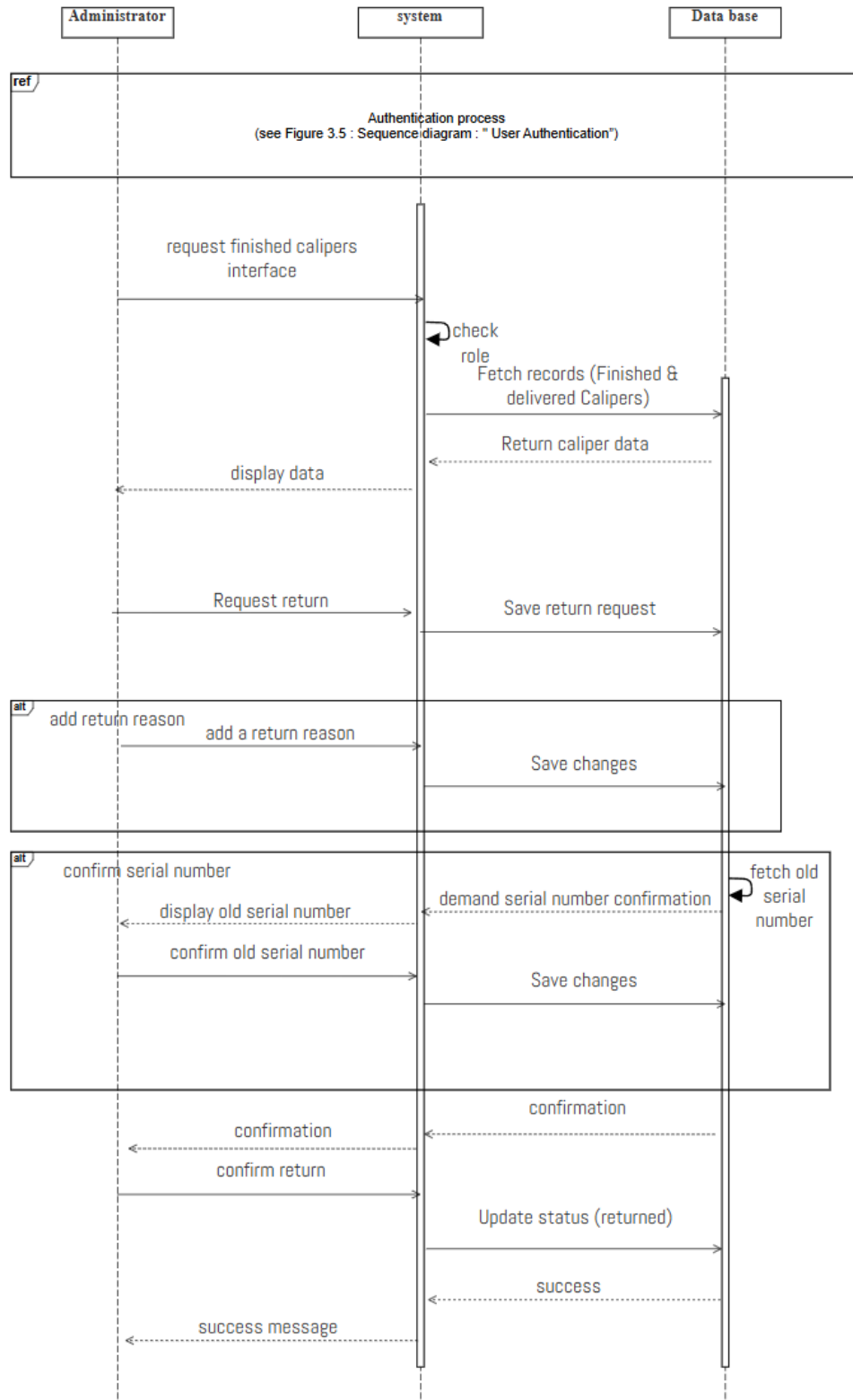


Figure 4.10: Sequence diagram: "Return Handling"

4.5 Interface Design and Prototyping

In this section, the GUI design will be discussed for the key functions implemented during Sprint 2. The screenshots below highlight the frontend interfaces used to manage the Caliper lifecycle.

4.5.1 Caliper Finalization Interfaces

Figure 4.11 presents the interface where a worker or administrator can review a caliper's details and officially mark the task as finished.

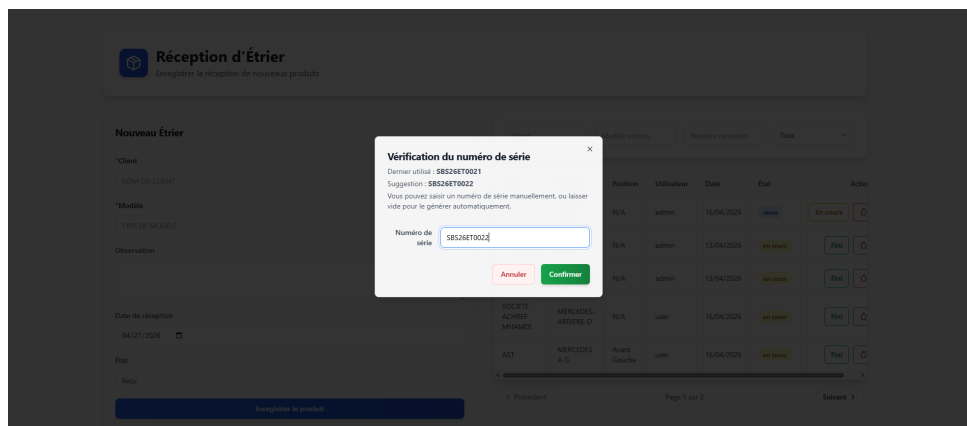


Figure 4.11: Caliper Finalization Interface

4.5.2 Finished Calipers Management Interfaces

The interface below illustrates the **Administrator's view** of the finished inventory. Unlike other roles, the administrator is granted full authority to manage finalized records, with the ability to modify serial numbers, process returns, or deliver the calipers .

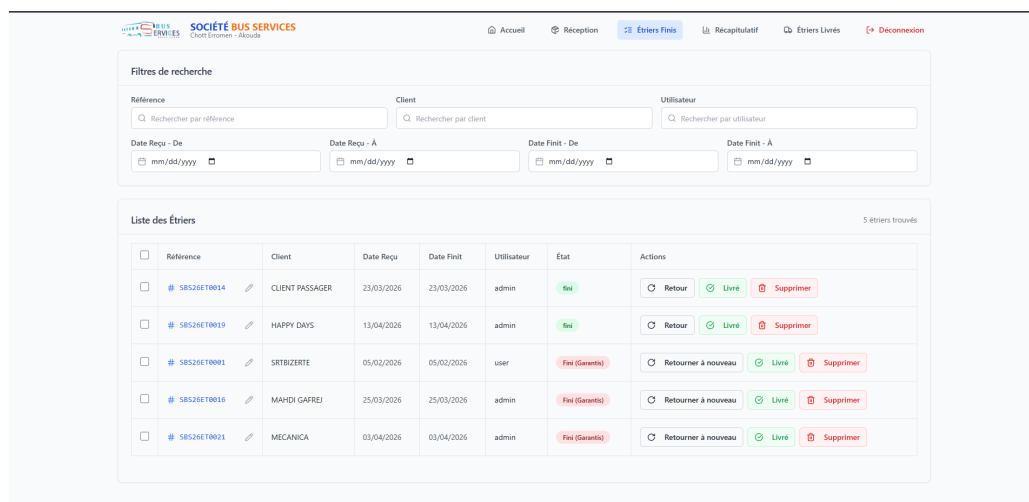


Figure 4.12: Finished Calipers Inventory - Administrator View

In contrast, the **Agent's interface** is strictly restricted to information retrieval. As shown below, agents can browse the inventory and view specific caliper details to track progress, but the administrative action buttons (Edit, Delete, Deliver) are disabled or hidden to maintain data integrity.

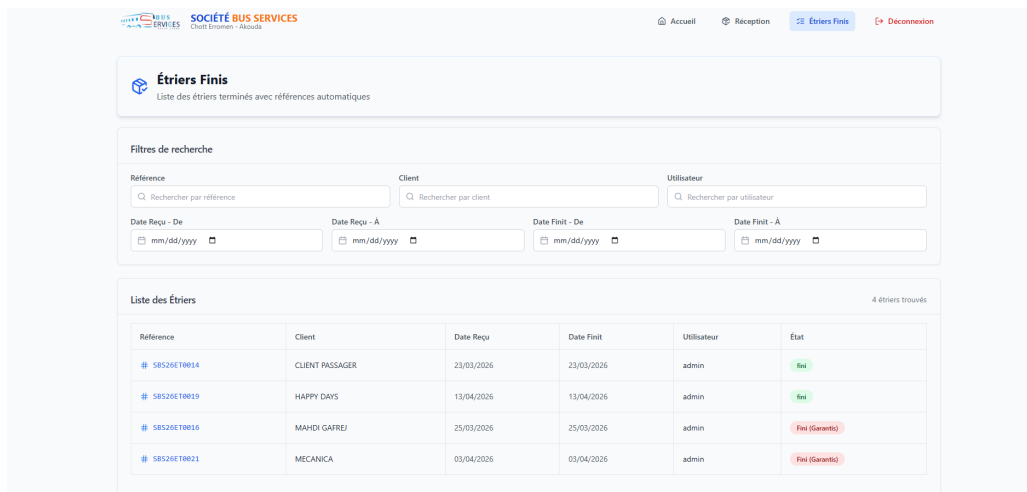


Figure 4.13: Finished Calipers Inventory - Agent Restricted View

An exclusive feature for the administrator is the **Print functionality**. As illustrated below, clicking the print icon triggers the system to generate a formal **Facture (Invoice)**, which includes the technical details of the calipers, the client information, and the final pricing for the service rendered.

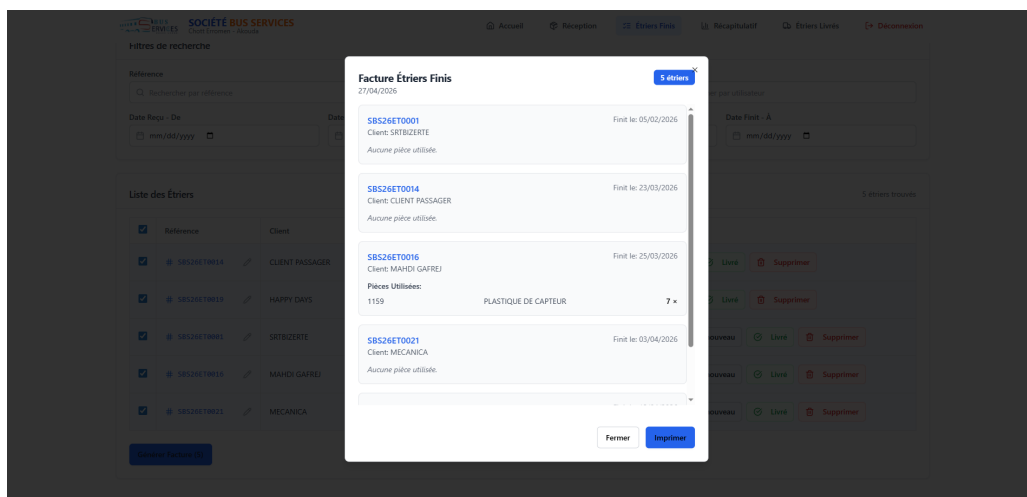


Figure 4.14: Administrative Feature - Facture (Invoice) Generation

4.5.3 Calipers Overview Interfaces

Figure 4.15 shows the administrative dashboard that provides a comprehensive recap of all calipers, allowing managers to monitor workflow and worker efficiency.

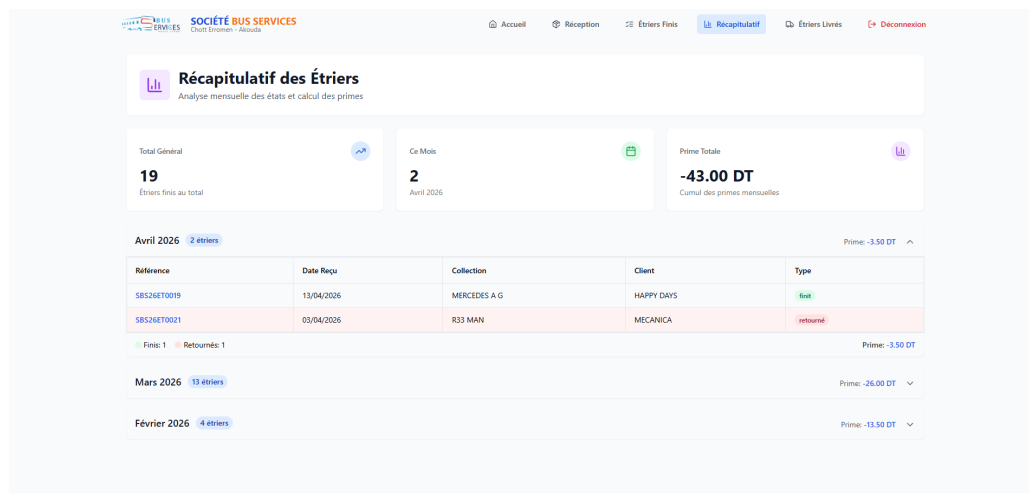


Figure 4.15: Administrative Overview Dashboard

4.5.4 Delivery Management Interfaces

Figure 4.16 allows users to select finished calipers and log their dispatch details to ensure accurate tracking of client deliveries.

Liste des Étriers Livrés (15 étriers trouvés)

Référence	Client	Utilisateur	Date Reçu	Date Finit	Action
# SBS26ET0001	SRTBIZERTE	user	05/02/2026	27/04/2026	🔍
# SBS26ET0011	SORETRAS	user	05/02/2026	27/04/2026	🔍
# SBS26ET0005	SOTREGAMES	admin	05/02/2026	27/04/2026	🔍
# SBS26ET0003	SOTREGAMES	user	18/02/2026	27/04/2026	🔍
# SBS26ET0006	MAHDI GAFREJ	admin	12/03/2026	27/04/2026	🔍
# SBS26ET0004	CLIENT PASSAGER	admin	23/03/2026	27/04/2026	🔍
# SBS26ET0007	SRTBIZERTE	admin	23/03/2026	27/04/2026	🔍
# SBS26ET0009	MONDE PIÈCES DE RECHANGE	user	23/03/2026	27/04/2026	🔍
# SBS26ET0008	MONDE PIÈCES DE RECHANGE	user	23/03/2026	27/04/2026	🔍
# SBS26ET0012	TRANSTU	admin	23/03/2026	27/04/2026	🔍

Figure 4.16: Delivery Management Interface

4.5.5 Return Handling Interfaces

The first stage of the process is the **Return Request** interface. As shown below, this form allows the user to initiate a return by selecting a delivered caliper and logging the specific justification or reason for the return.

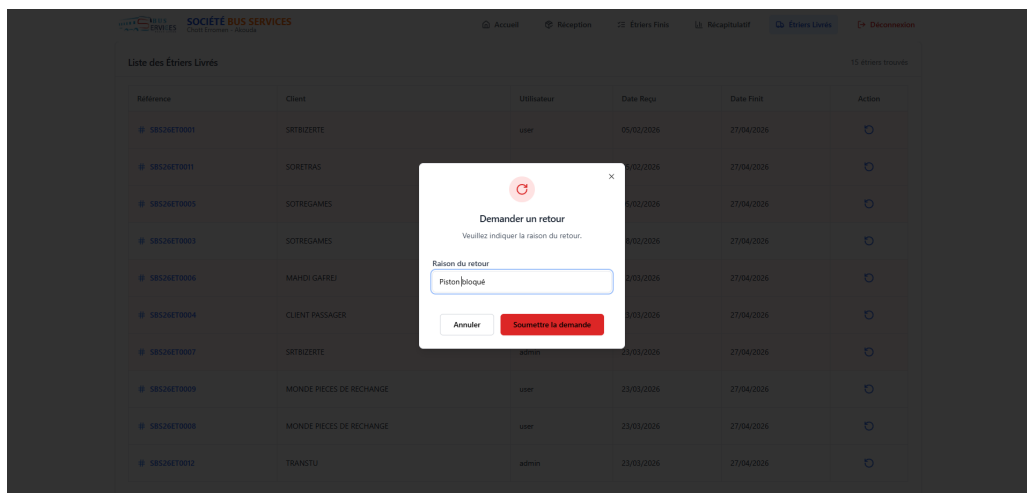


Figure 4.17: Return Handling - Initial Request Form

Once the request is initiated, the system requires the user to **Confirm the Old Serial Number**. This security step ensures that the physical component being returned matches the digital record in the database before any status changes are applied.

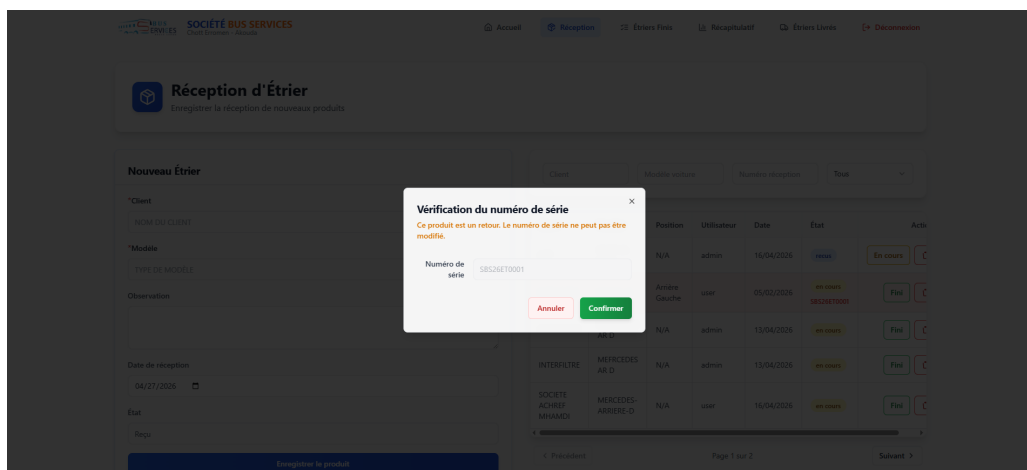


Figure 4.18: Return Handling - Serial Number Verification

Upon successful verification, the system displays a **Return Completed** confirmation. This informs the operator that the transaction has been logged and the database has been successfully updated.

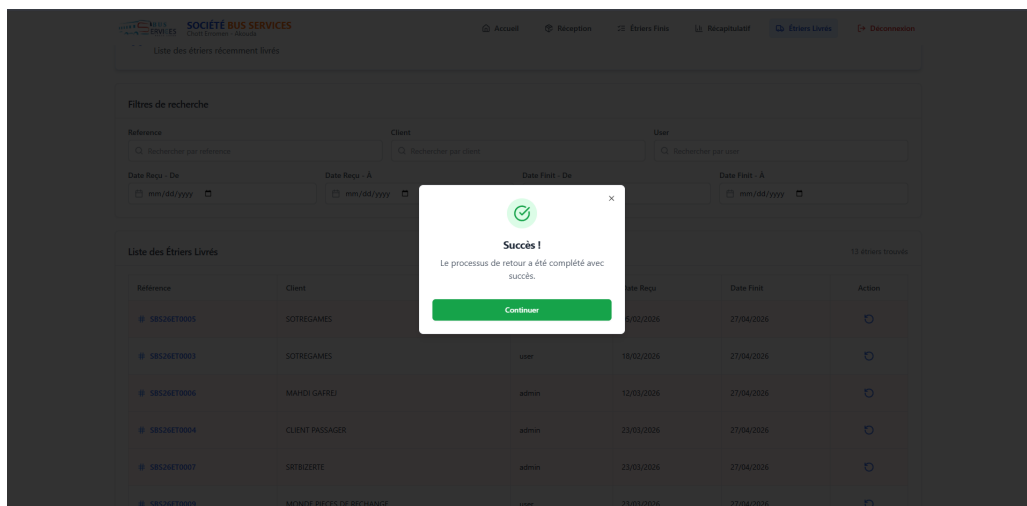


Figure 4.19: Return Process Success Confirmation

As illustrated in the interface below, the caliper is then automatically re-inserted into the active production list with the specific status **”Returned Insurance”**. This ensures the item is prioritized for inspection and rework.

The screenshot shows a web interface for managing Caliper items. At the top, there are four filter buttons: 'Client', 'Modèle voiture', 'Numéro réception', and 'Tous' (with a dropdown arrow). Below the filters is a table with the following columns: Client, Modèle, Position, Utilisateur, Date, État, and Actions.

Client	Modèle	Position	Utilisateur	Date	État	Actions
AST	KNOR-D-MAN	N/A	admin	16/04/2026	recus	En cours
HAPPY DAYS	MERCEDES-ARRIERE-D	Avant Droit	admin	05/05/2026	recus	En cours
HAPPY DAYS	MERCEDES A G	N/A	admin	05/05/2026	recus	En cours
SRTBIZERTE	VOLVO -G	Arrière Gauche	user	05/02/2026	en cours SBS26ET0001	Finis
INTERFILTRE	MEFRCEDES AR D	N/A	admin	13/04/2026	en cours	Finis
INTERFILTRE	MEFRCEDES AR D	N/A	admin	13/04/2026	en cours	Finis
SOCIETE ACHREF MHAMDI	MERCEDES-ARRIERE-D	N/A	user	16/04/2026	en cours	Finis
AST	MERCEDES A G	Avant Gauche	user	16/04/2026	en cours	Finis
MRS	MERCEDES A G	Avant Gauche	user	16/04/2026	en cours	Finis

At the bottom of the table, there are navigation buttons: '< Précédent', 'Page 1 sur 2', and 'Suivant >'.

Figure 4.20: Caliper in "Returned Insurance" State

4.6 Conclusion

In Conclusion during this second sprint, it was possible to achieve the implementation of the basic functional life cycle of the calipers. Designing interfaces for finalization, delivery, and returns processing allowed us to ensure a comprehensive workflow to maintain data integrity and security based on role permissions. Now, administrators have the opportunity to conduct high-level management and document financial operations with the help of an invoice generator, while agents can carry out technical operations and manage the "Returned Insurance" life cycle.

Having completed this life cycle, it is possible to provide a base for future work. In the next chapter **Sprint 3: Advanced Parts Management and AI Integration** – it is going to be discussed how to handle separate parts and integrate AI into the software.

CHAPTER 5

Sprint 3: Parts Management and AI Integration

Chapter Contents

5.1 Introduction	68
5.2 Sprint Backlog: Sprint 3	68
5.3 Use Case Diagrams	69
5.4 Sequence Diagrams	72
5.5 Interface Design and Prototyping	75
5.6 Conclusion	81

Chapter 5

Sprint 3: Parts Management and AI Integration

5.1 Introduction

This third sprint focuses on enhancing the system’s technical capabilities by managing individual spare parts (pieces) and integrating intelligent features. This phase is critical for automating manual tasks, such as component identification and data entry, through the use of Artificial Intelligence to ensure a modern, high-efficiency workflow.

5.2 Sprint Backlog: Sprint 3

Table 5.1 presents the backlog for Sprint 3, outlining the functionalities, user stories, and their respective priorities.

Functionality	User Story	Priority
Spare Parts Management	As an administrator, I want to manage the catalog of spare parts so that they can be assigned to caliper repairs.	High
AI Piece Recognition	As an agent, I want to photograph a part so the AI can automatically identify its model and specifications.	Low
Voice Command Entry	As an agent, I want to use voice commands to create records so that I can work hands-free in the workshop.	Low

Table 5.1: Sprint Backlog: Sprint 3

5.3 UML Diagrams Design

This section describes the Use Case and Sequence Diagrams designed to handle the logic of parts management and AI interaction.

5.3.1 Use Case Diagram: Spare Parts Management

The following Figure 5.1 and Table 5.2 illustrate how administrators manage the spare parts catalog within the system.

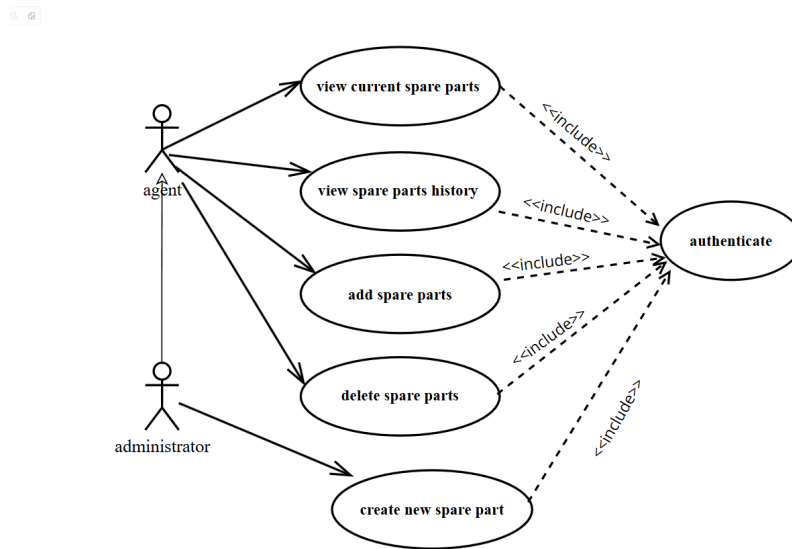


Figure 5.1: Use case "Spare Parts Management"

Use Case	Spare Parts Management
Actor	Administrator, Agent
Pre-condition	The user must be authenticated. The caliper must be in the 'in progress' state.
Post-condition	The user is granted access to add spare parts into calipers
Main Scenario	1. The user opens the reception interface 2. The system displays the current list of calipers 3. The user selects "in progress" caliper 4. the system displays the current list of spare parts and history parts 5. The user adds to current spare parts of the caliper 6. The user deletes from current spare parts of the caliper 7. The admin create a new spare part 8. The system saves changes
Exception	Invalid Data: Error message if required fields (e.g., 'Reference article') are missing.

Table 5.2: Use Case Description – Spare Parts Management

5.3.2 Use Case Diagram: AI Piece Recognition

Figure 5.2 and Table 5.3 illustrate the process of using the integrated AI model to recognize parts via the mobile camera.

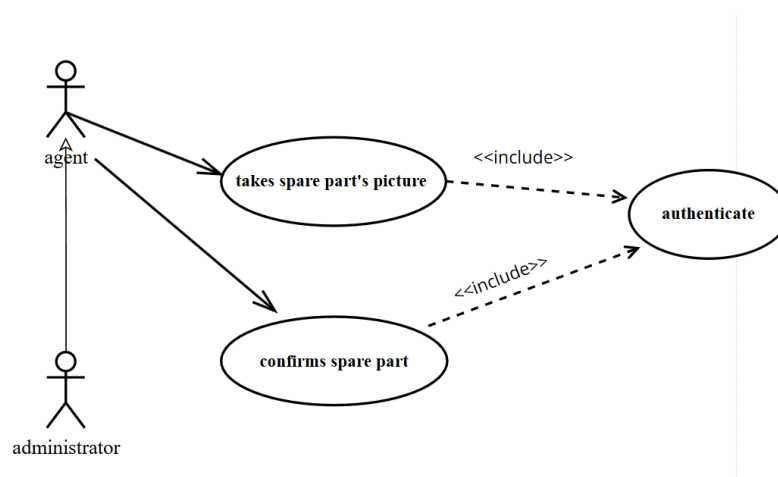


Figure 5.2: Use case "AI Piece Recognition"

Use Case	AI Piece Recognition
Actor	Agent, Administrator
Pre-condition	The user must have camera permissions enabled and be authenticated. The caliper must be in the 'in progress' state.
Post-condition	The system identifies the part and suggest the correct spare part.
Main Scenario	1. The user opens the reception interface 2. The system displays the current list of calipers 3. The user selects "in progress" caliper 4. The user opens the recognition module 5. The user captures a photo of the physical part 6. The system sends the image to the AI model 7. The system returns the most likely part match 8. The user confirms the correct part to link it to the caliper record. 9. The system saves changes

Table 5.3: Use Case Description – AI Piece Recognition

5.3.3 Use Case Diagram: Voice Command Entry

Figure 5.3 and Table 5.4 outline the hands-free operation mode for creating records.

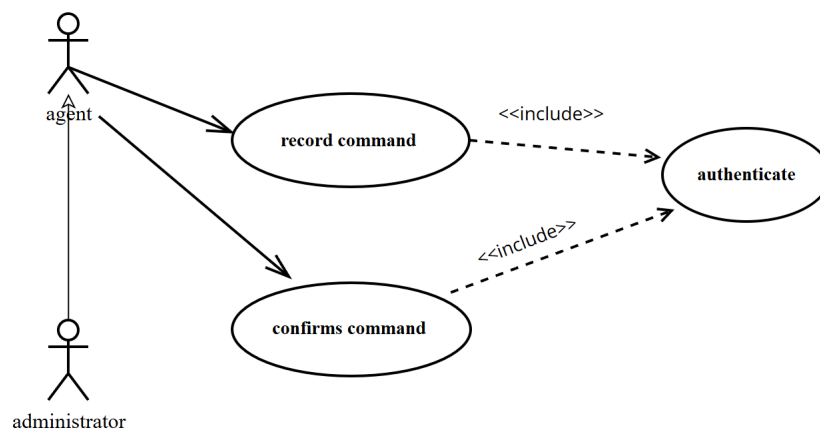


Figure 5.3: Use case "Voice Command Entry"

Use Case	Voice Command Entry
Actor	Administrator, Agent
Pre-condition	The user must have microphone permissions enabled and be authenticated. The caliper must be in the 'in progress' state.
Post-condition	A new query is successfully drafted based on the voice input.
Main Scenario	1. The user opens the reception interface 2. The user activates the voice command mode 3. The system prompts the user to speak 4. The user dictates the part details or actions 5. The system processes the audio, transcribing it into text 6. The system analyzes the transcribed text to determine the user's intent and translates it into a structured query 7. The system automatically run the query 8. The user reviews and confirms the generated data
Exception	Audio Unclear: The system displays an error if the speech cannot be processed and prompts the user to repeat. Intent Unrecognized: The system prompts the user to rephrase if it cannot determine a valid query from the dictation.

Table 5.4: Use Case Description – Voice Command Entry

5.4 Sequence Diagrams

5.4.1 Spare Parts Management

Figure 5.4 presents the sequence diagram illustrating the interactions required to manage the spare parts inventory.

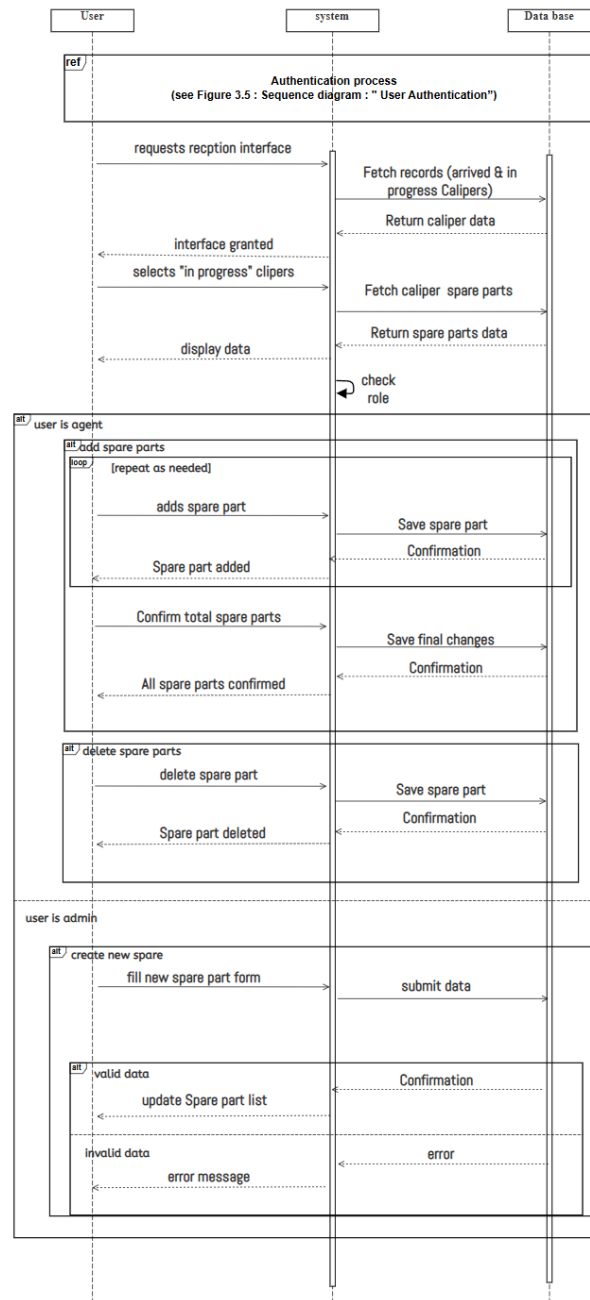


Figure 5.4: Sequence diagram: "Spare Parts Management"

5.4.2 AI Piece Recognition

Figure 5.5 illustrates the step-by-step communication between the mobile user, the system, the AI model, the Date base for spare parts identification.

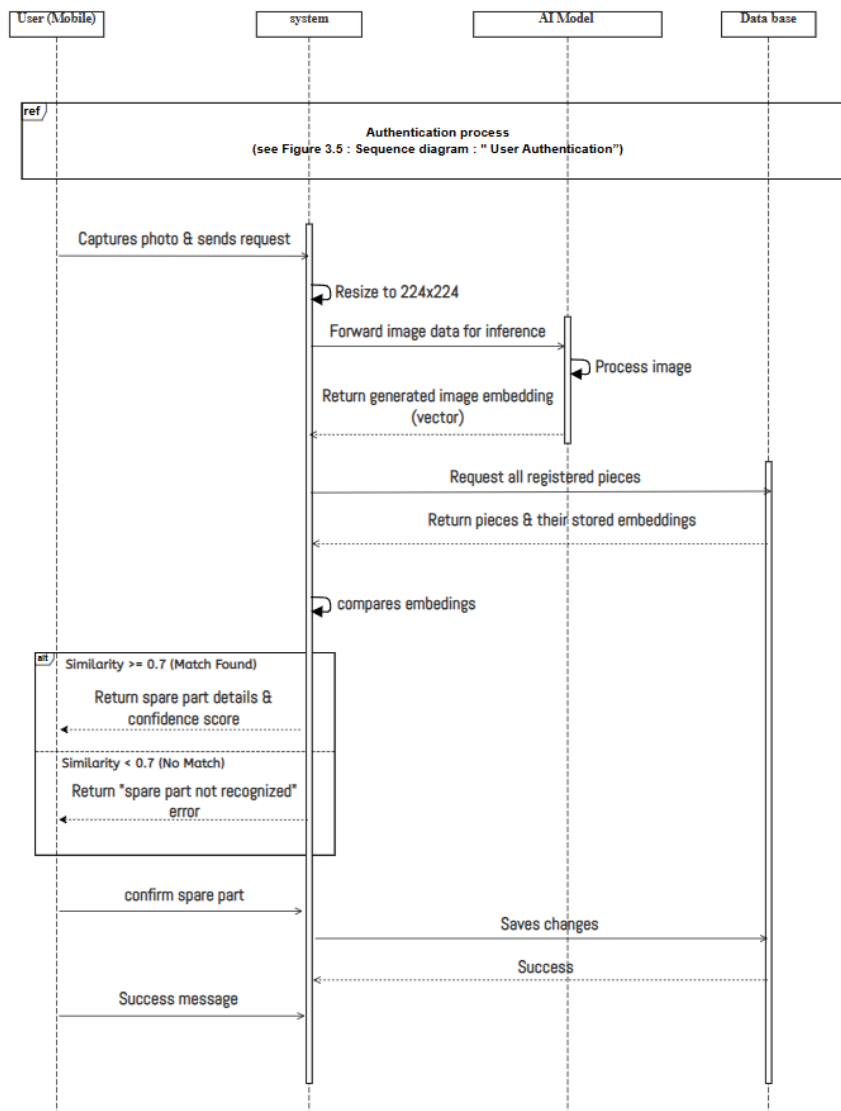


Figure 5.5: Sequence diagram: "AI Recognition Logic"

5.4.3 Voice Command Entry

Figure 5.6 details the sequence of events when an agent utilizes the speech-to-text service to draft a new record.

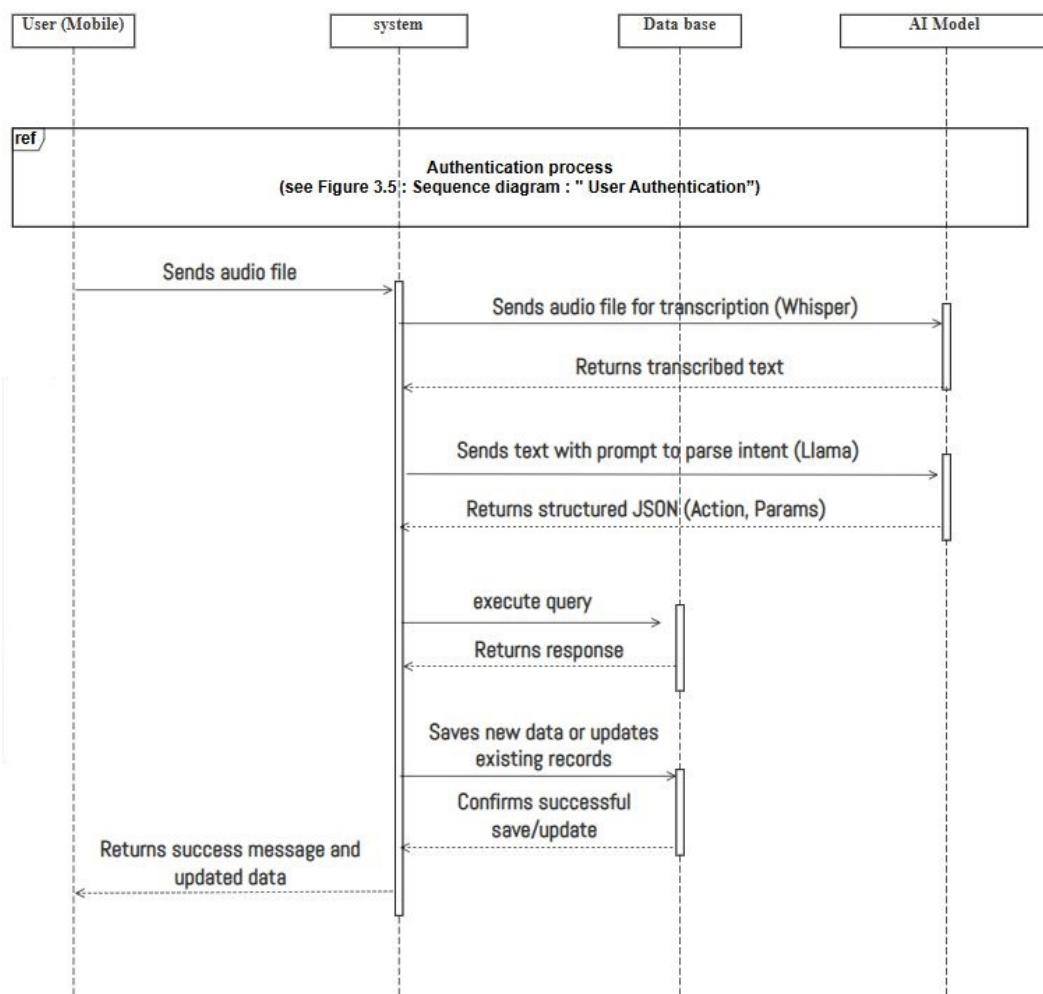


Figure 5.6: Sequence diagram: "Voice Command Entry Logic"

5.5 Interface Design and Prototyping

This section presents the GUI prototypes for modules developed during this sprint.

5.5.1 Spare Parts Management Interfaces

This interface allows users to assign and add specific spare parts to a caliper during the repair or assembly process, ensuring accurate tracking of used materials.

The screenshot shows the 'Réception d'Étrier' (Caliper Reception) interface. A modal window titled 'Gestion des pièces' is open, allowing users to manage parts. It includes a search bar, a quantity input (set to 1), and an 'Ajouter' button. Below, there's a table with columns 'Référence', 'Désignation', and 'Quantité'. The table lists two items: '1040 KIT KNOOR SNG/SN7/SK7/NA7/ST6/ST7' with a quantity of 3, and '1560 SOUFLET KNOOR SNS OTOKAR AV' with a quantity of 4. At the bottom of the modal are buttons for '+ Nouvelle Pièce', 'Annuler', and 'Enregistrer'. The background shows a form for 'Nouveau Étrier' with fields for client, model, observation, date, and status, along with a table of historical data.

Figure 5.7: Spare Part Assignment Form

Figure 5.8 displays the historical log of spare parts associated with a specific caliper. This interface is especially crucial when a caliper is returned, allowing technicians to review past modifications and replacements.

This screenshot shows the same 'Gestion des pièces' modal, but with the 'HISTORIQUE DES RETOURS (2 CYCLES)' section expanded. It displays two return cycles: 'Retour #2' dated 30/03/2026 (1 pièce) and 'Retour #1' dated 30/03/2026 (2 pièces). Below these, a table lists the parts returned: '1001 TAPPET KNOOR SNS ZEP0016909' with a quantity of 1, and '1004 CALIPER CAP SET' with a quantity of 1. The modal also shows the '+ Nouvelle Pièce', 'Annuler', and 'Enregistrer' buttons. The background interface remains the same as in Figure 5.7.

Figure 5.8: Spare Part History Overview

The interfaces below are exclusively utilized by administrators to register entirely new spare parts into the system's global catalogue. The system incorporates real-time form validation, ensuring that if any field is left empty or is invalid, the user is immediately prompted with an error message to correct their entry.

The screenshot shows the 'Ajouter une pièce' (Add a part) form in the SOCIÉTÉ BUS SERVICES admin interface. The form is titled 'Ajouter une pièce' with a close button (X). It contains three input fields: 'Nom de la pièce' (Part name), 'Référence Article' (Article reference) with the example 'Ex: REF-12345', and 'Code Barre' (Barcode) with the label 'Numéro de code barre'. A blue button at the bottom is labeled 'Ajouter la pièce'.

Figure 5.9: New Spare Part Registration

The screenshot shows the 'Ajouter une pièce' form with an error message displayed at the top: 'Cette référence article existe déjà' (This article reference already exists). The form fields are filled with 'Test' for 'Désignation', '1001' for 'Référence Article', and '1001' for 'Code Barre'. A blue button at the bottom is labeled 'Ajouter la pièce'.

Figure 5.10: Registration Form - Error Handling

5.5.2 AI Piece Recognition Interfaces

The interface will act as the primary starting point for identifying the part, having a prominent camera icon which allows the user to easily start the capturing process. After processing the captured image using the AI algorithm, the interface will present the identified part to the user for verification before securely saving it to the system.

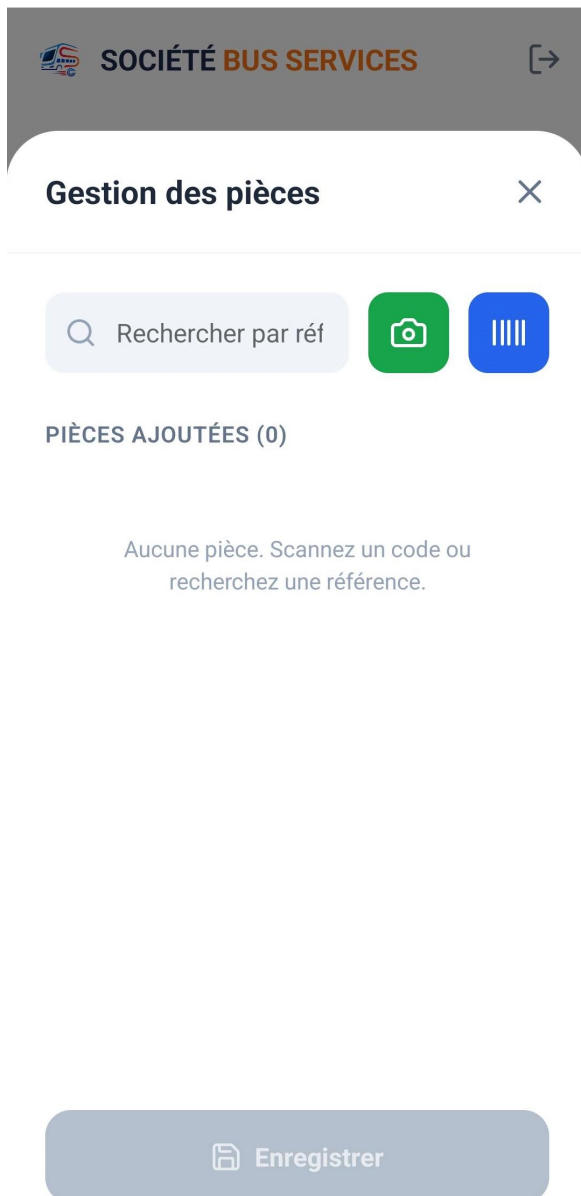


Figure 5.11: AI Recognition - Camera Access

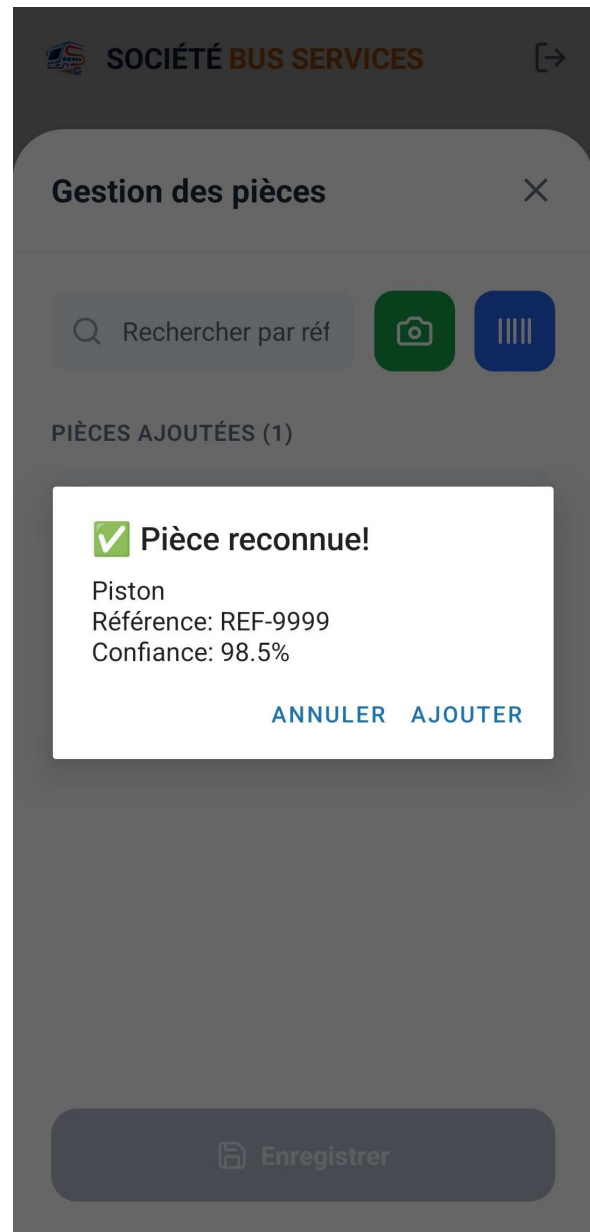


Figure 5.12: AI Recognition - Result Confirmation

5.5.3 Voice Command Entry Interfaces

This interface activates when the hands-free mode is engaged, providing visual feedback to the agent that the system is actively listening and processing speech.

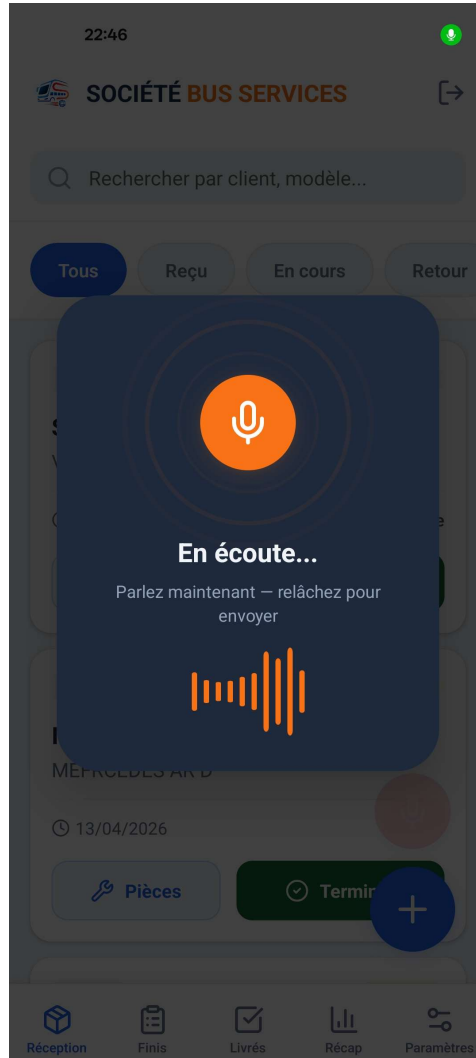


Figure 5.13: Voice Command - Active Recording Interface

Upon successfully processing the audio and executing the generated query, the system provides a visual success message to confirm to the agent that the requested action was accurately interpreted and completed. Conversely, if the speech is unclear or the system cannot determine a valid structured query from the dictation, an error handling interface is displayed to notify the user.

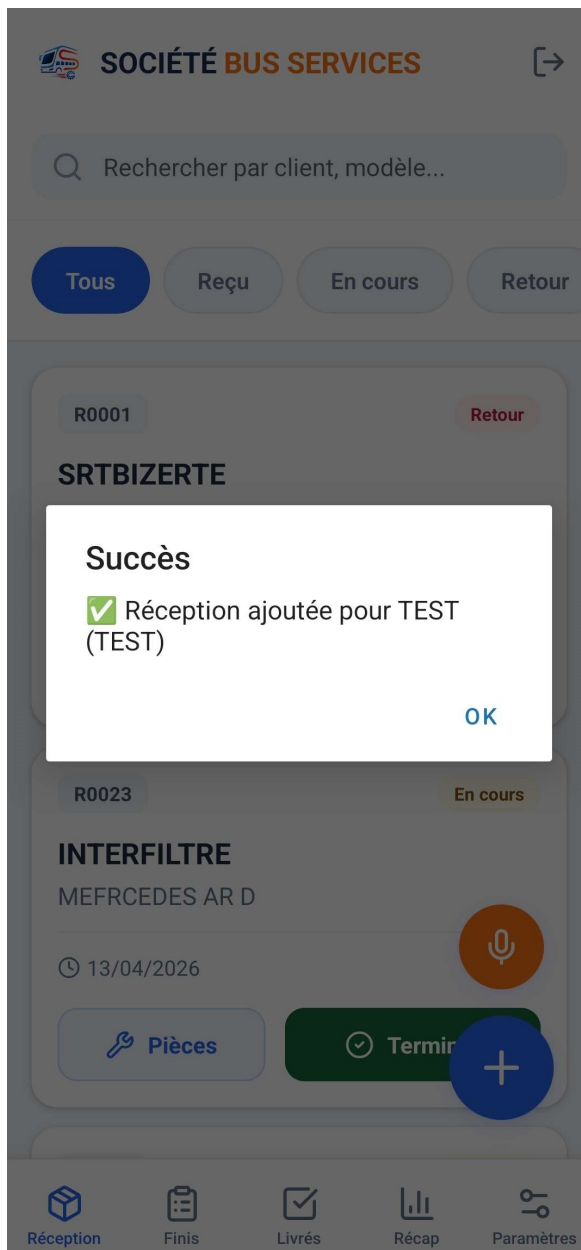


Figure 5.14: Voice Command - Query Success Message

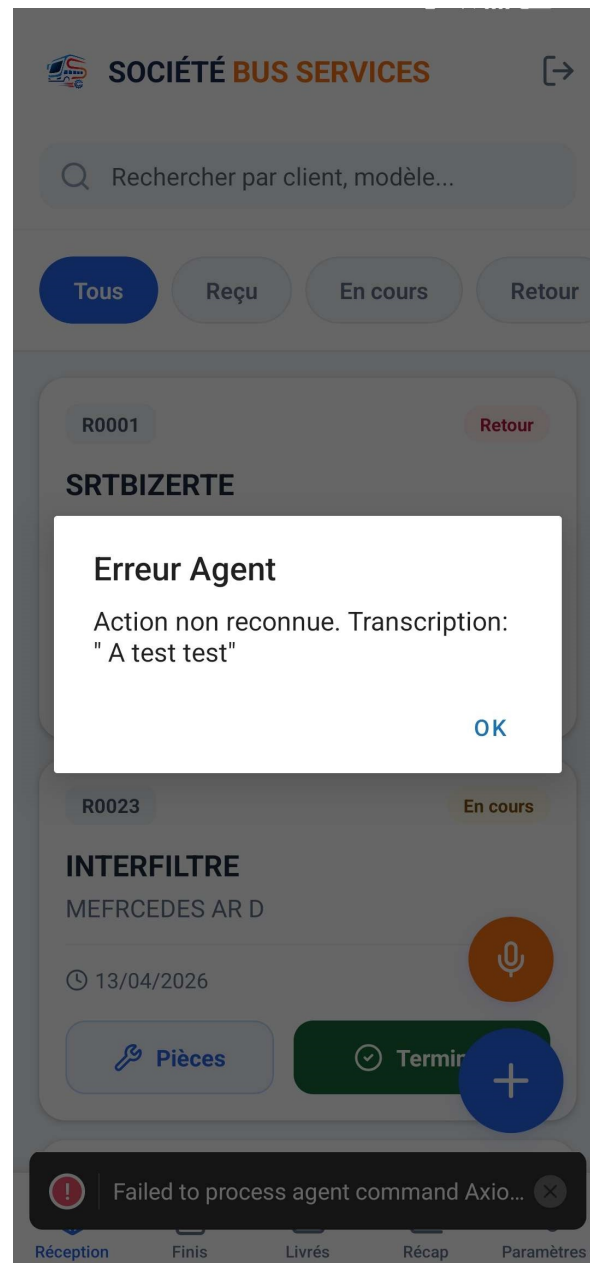


Figure 5.15: Voice Command - Error Handling Interface

5.6 Conclusion

This sprint successfully expanded the system's capabilities by introducing Spare Parts Management alongside intelligent, hands-free features. The integration of AI Piece Recognition and Voice Command Entry significantly reduces manual data input, streamlining the workshop workflow for agents while giving administrators robust control over the parts catalog.

General Conclusion

This project was born from a real need to simplify and automate daily operations within industrial environments, particularly for companies like **BUS SOFTWARE SERVICES**. Traditional tools often fall short by being highly susceptible to human error, or simply ill-adapted to the fast-paced and evolving nature of industrial work. That is why we set out to build a centralized web and mobile platform that brings everything together in one place: efficiently managing brake calipers, tracking spare parts, and evaluating employee performance.

We approached the development across three focused sprints. The first sprint established the core foundation of the system: secure user authentication, account management, calipers reception, and a data import engine allowing administrators to populate the catalogue via Excel uploads or automated server synchronization. The second sprint tackled the complete caliper lifecycle, covering finalization, finished calipers management, an administrative overview dashboard, delivery management, and return handling.

The third sprint extended the platform with advanced technological capabilities: a spare parts catalog management system, AI-powered piece recognition via the mobile camera, and a voice command entry module enabling hands-free data input directly on the workshop floor.

The platform was engineered using a robust, modern technology stack comprising React, React Native, ExpressJS, MongoDB and FastAPI. These frameworks were explicitly chosen to ensure that the system is not only powerful and responsive but also easy to use and scalable as the company's needs change.

Future Perspectives

Looking ahead, we see exciting opportunities to take this platform even further. We plan to add:

- **Continuous AI Model Training:** Expanding the current image recognition dataset. While the artificial intelligence successfully detects several key spare parts, continuous training will allow the model to automatically recognize a comprehensive catalog of bus engine components with higher precision.
- **System-Wide Voice Command Integration:** Extending the voice-to-text mod-

ule beyond the initial reception phase. The goal is to integrate hands-free data entry across all application workflows and actions, enabling technicians to fully operate the app without interrupting their physical tasks.

These additions ensure the platform remains adaptable alongside its users. Ultimately, this project provided a scalable industrial management foundation while allowing us to apply software engineering in a real-world enterprise.

Bibliography

- [1] **BDPedia. Understanding MVC Architecture.** <http://orm.bdpedia.fr/mvc.html>. Last consulted: 12/02/2026.
- [2] **Microsoft. Visual Studio Code - Basics.** <https://code.visualstudio.com/docs/introvideos/basics>. Last consulted: 04/01/2026.
- [3] **PM Partners. The Agile Journey – A Scrum Overview.** <https://www.pm-partners.com.au/insights/the-agile-journey-a-scrum-overview/>. Last consulted: 07/03/2026.
- [4] **StarUML Docs. Working with UML Class Diagrams.** <https://docs.staruml.io/working-with-uml-diagrams/class-diagram>. Last consulted: 09/04/2025.
- [5] **React Documentation. Learn React.** <https://react.dev/learn>. Last consulted: 05/01/2026.
- [6] **MongoDB. MongoDB Documentation.** <https://www.mongodb.com/docs/>. Last consulted: 01/02/2026.

Abstract

This report details a web and mobile platform developed for **BUS SOFTWARE SERVICES** to automate bus brake caliper management. Built with MERN and FastAPI, it handles the entire caliper lifecycle. Integrating AI part recognition and voice commands, the system significantly improves industrial workflow efficiency and precision.

Key words: Web Application, Mobile Application, Brake Caliper, MERN, Artificial Intelligence, Voice Command.

Résumé

Ce rapport présente une plateforme web et mobile développée pour **BUS SOFTWARE SERVICES** afin d'automatiser la gestion des étriers de freins. Conçu avec MERN et FastAPI, le système optimise tout le cycle de vie des étriers. En intégrant l'IA et des commandes vocales, il améliore considérablement l'efficacité et la précision des flux de travail industriels.

Mots clés: Application Web, Application Mobile, Étriers, MERN, Intelligence Artificielle, Commande Vocale.

الملخص

يقدم هذا التقرير منصة ويب وتطبيقاً هاتفياً تم تطويرهما لصالح **SERVICES SOFTWARE BUS** لأتمتة إدارة أقراص فرامل الحافلات. تم بناء النظام باستخدام تقنيات MERN و FastAPI لإدارة دورة حياة الأقراص بالكامل. بفضل دمج الذكاء الاصطناعي للتعرف على القطع والأوامر الصوتية، يحسن النظام كفاءة ودقة العمليات الصناعية بشكل ملحوظ.

الكلمات المفتاحية: تطبيق ويب، تطبيق الهاتف المحمول، مكبس الفرامل، MERN، الذكاء الاصطناعي، الأوامر الصوتية.